دليل مذاكرة عملي · بالعامية المصري

Docker

من الفكرة لحد ما تقوّم Container على السيرفر

ليه Docker اتعمل أصلاً، الـ Images والـ Containers، هيكل الـ Layers، الـ Volumes والـ Networks، كتابة Dockerfile صح، Multi-stage Build، Docker Hub و Docker Compose.



```
docker build -t app .
```

```
docker run -d -p 8080:80 nginx
```

```
FROM node:18 AS builder
```

```
docker volume ls
```

```
docker compose up -d
```

إعداد

MOAZ ABDEL-FATTAH

نسخة مُعاد تصميمها

+ فصول إضافية و Cheat Sheet

المحتويات

الكتاب ماشي بالترتيب اللي هتحتاجه وانت بتتعلم: نفهم المشكلة الأول، وبعدين نشتغل على Images جاهزة، وبعدها نبني بتاعتنا. الفصول اللي عليها علامة إضافة اتضافت علشان الكتاب يبقى كامل.

إزاي تذاكر من الكتاب ده؟ دليل الألوان

- **معلومة مهمة**: حاجة لازم تفضل فاكرها
- **Best Practice**: الطريقة الصح في الشغل الحقيقي
- **خلي بالك**: غلطة شائعة أو فخ
- **مثال عملي**: جرّبه بإيدك على جهازك

01 المقدمة: ليه Docker اتعمل أصلاً؟ 4

1.1 المشكلة الأولى: الـ Compatibility بين السيرفر والـ Apps 4
1.2 المشكلة التانية: كدا App على نفس السيرفر 5
1.3 البداية: فكرة الـ chroot 6
1.4 الحل الأول: Virtual Machines 6
1.5 الحل التاني: Linux Containers 6
1.6 لكن الحلو مبيكملش: مشاكل الـ Linux Containers 8

02 Docker: الفكرة وطريقة الشغل 10

2.1 Docker Containers 10
2.2 المشكلة الأولى اتحلّت إزاي؟ 10
2.3 الرحلة من الـ Dockerfile لحد الـ Container 11
2.4 خطة الجزء العملي 12

03 الـ Images وإزاي بنتعامل معاها 13

3.1 يعني إيه Image؟ 13
3.2 بنجيب الـ Image منين؟ (docker pull) 13
3.3 أعرف إيه الـ Images اللي نازلة عندي؟ 14
3.4 حذف Image 15

04 تشغيل الـ Images والتعامل مع الـ Containers 16

4.1 تشغيل Container (docker run) 16
4.2 ليه الـ Container اشتغل وخلص على طول؟ 17
4.3 عرض الـ Containers (docker ps) 17
4.4 أوامر التعامل مع الـ Container 18
4.5 مثال عملي: busybox 18
4.6 مثال عملي: nginx Web Server 20
4.7 خصائص الـ Container في سطور 21

05 مراقبة وفحص الـ Containers: logs و inspect و stats إضافة 23

5.1 docker logs: الـ Container بيقول إيه؟ 23
5.2 docker inspect: كل التفاصيل عن أي حاجة 24
5.3 docker stats: بياكل موارد قد إيه؟ 24
5.4 أوامر مساعدة مفيدة 25

06 فهم Structure الـ Image: الـ Layers 26

6.1 الـ Image مكوّنة من Layers 26
6.2 الـ Container Layer مؤقتة (Volatile) 27
6.3 لو عملت ملف جديد جوه الـ Container؟ 27
6.4 لو عدّلت ملف موجود؟ (Copy-on-Write) 27
6.5 لو مسحت ملف موجود؟ (Whiteout) 28

07 الـ Volumes: البيانات اللي لازم تعيش 30
7.1 ليه محتاجين Volumes؟ 30
7.2 يعني إيه Volume في Docker؟ 30
7.3 أنواع الـ Volumes 31
7.4 أوامر إدارة الـ Volumes 34
7.5 لو كذا Container عدّلوا في نفس الوقت؟ 34
7.6 إزاي المهندسين بيحلّوا المشكلة دي؟ 35

08 Docker Networks: إزاي الـ Containers بتكلّم بعض إضافة 37
8.1 يعني إيه Docker Network؟ 37
8.2 أنواع الشبكات: bridge و host و none 37
8.3 الشبكات اللي بتعملها بنفسك (User-defined) 38
8.4 الـ DNS جوه الشبكة: كلّم الـ Container باسمه 38

09 الـ Dockerfile: وصفة بناء الـ Image 41
9.1 يعني إيه Dockerfile؟ 41
9.2 بنستخدم Dockerfile ليه؟ 41
9.3 أهمية الـ Dockerfile: توثيق مش بس بناء 42
9.4 إزاي تنشئ Dockerfile 42
9.5 الشكل العام لـ Dockerfile 42
9.6 FROM: نقطة البداية 43
9.7 LABEL: بيانات وصفية 43
9.8 ENV و ARG: المتغيرات 44
9.9 WORKDIR: مسار الشغل 45
9.10 COPY و ADD: نقل الملفات 45
9.11 RUN: عضلات الـ Dockerfile 47
9.12 CMD و ENTRYPOINT: أمر التشغيل 47
9.13 EXPOSE: يافطة معلومات 49
9.14 USER: متشغّلش كـ root 49

10 بناء الـ Image و Multi-stage Build 51
10.1 أمر docker build 51
10.2 مثال عملي: موقع static بـ nginx 51
10.3 المشكلة: الـ Image المليانة كراكيب 52
10.4 الحل: Multi-stage Build (نظرية المطبخ وصالة الطعام) 53
10.5 مثال عملي متكامل: تطبيق React مع Nginx 53
10.6 تحليل السحر اللي حصل هنا 54
10.7 إيه المكاسب اللي طلعنا بيها؟ 54

11 ملف .dockerignore: متبعتش الكراكيب لـ build إضافة 56
11.1 المشكلة: الـ Build Context بيتبعت كله 56
11.2 الحل: ملف .dockerignore 56
11.3 قواعد الكتابة 57
11.4 المكاسب 57

12 رفع الـ Image على Docker Hub: tag و login و push إضافة 59
12.1 اسم الـ Image لازم يقول هي رايحة فين 59
12.2 الخطوات: tag ثم login ثم push 59
12.3 Registries تانية غير Docker Hub 60

13 Docker Compose: شغّل المشروع كله بأمر واحد إضافة 62
13.1 المشكلة: أوامر كتير لمشروع واحد 62
13.2 ملف compose.yaml 62
13.3 أوامر Compose الأساسية 64

أ Cheat Sheet: كل الأوامر في مكان واحد 67

ب إجابات أسئلة المراجعة 70

01

الفصل الأول

المقدمة: ليه Docker اتعمل أصلاً؟

عندنا كام مشكلة عايزين نفهمهم، وإزاي اتحلّوا على مر السنين، وإيه دور Docker في ده. الفصل ده هو الأساس اللي هتبني عليه كل اللي جاي.

المشكلة الأولى: الـ Compatibility بين السيرفر والـ Apps
المشكلة التانية: كذا App على نفس السيرفر
البداية: فكرة الـ chroot
الحل الأول: Virtual Machines
الحل التاني: Linux Containers
لكن الحلو مبيكملش: مشاكل الـ Linux Containers

الكتاب ده بيبني على اللي فات

هتحتاج من الكتب اللي قبله: الـ **Processes** و `ps` (كتاب الـ Terminal فصل 9)، والـ **Ports** و **localhost** (كتاب الإنترنت فصل 3)، و `npm install` و `npm run build` (كتاب الفرونت إند فصول 13 و 18)، و YAML (كتاب APIs و JSON فصل 21). والأمثلة هنا مكتوبة على سيرفر Linux (`moaz@Linux`)، بس نفس الأوامر بالحرف بتشتغل على الماك، وفيه صندوق في فصل 3 بيقولك الفروق.

1.1 المشكلة الأولى: الـ Compatibility بين السيرفر والـ Apps

قبل كده لو عندك مشروع كبير فيه موقع وموبايل أبلكيشن وباك إند كمان، فكانت الدنيا بتمشي إزاي؟ مين بيتحط على السيرفر ومين بيروح فين؟ خلينا نقسمها 3 حاجات:

(1) كود الـ Front end

بعد ما الشغل فيه يخلص، المبرمج بيشغل الأمر `npm run build` يحوّله لنسخة production ويطلع folder فيه:

- `index.html`
- ملفات JS مضغوطة
- ملفات CSS مضغوطة
- الـ assets

وده اللي بيترفع على السيرفر، وبنشغّل Nginx على الـ folder ده عشان يشتغل كـ Web Server ويقدّم ملفات الموقع للمستخدمين.

(2) كود الـ Back end

بعد ما الشغل فيه يخلص، المبرمج بيشغل الأمر `npm install` علشان ينزّل الـ dependencies ويبعت الملفات على السيرفر.

(3) كود الـ Mobile app

ده كان أسهلهم، لأنك بمجرد ما تعمل export وتطلع نسخة APK أي حد يقدر ينزّلها على الموبايل ويثبّتها مباشرة. مش محتاج dependencies ومش بيتحط على سيرفر بنفس الطريقة، فمش هنركز عليه.

وهنا بقى بتحصل المشكلة

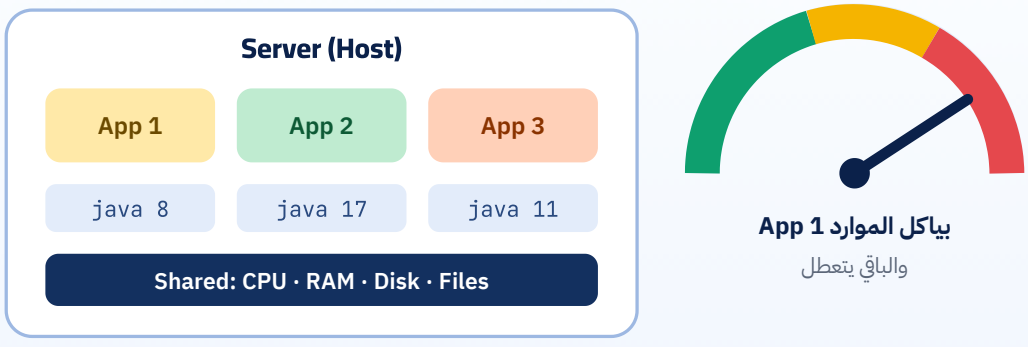
- الكودين (الفرونت والباك) دول ليهم dependencies خاصة بيهم زي node و java و python وغيرهم، فلازم تكون متسطبة على السيرفر أصلاً، وكمان بالـ Versions الصح عشان يشتغلوا.
- فلما يحصل نقص في أي حاجة من الحاجات دي أو اختلاف في الـ Versions، الكود ميشتغلش. ولما تيجي تكلم الـ Developer هيقولك الجملة المشهورة: **"It works on my machine"**، يعني "دي شغالة عندي عادي وسليمة!"، فلازم تراجع كل الحاجات الناقصة وتسطّبهم وتمشّي حالك.

فإحنا هنا محتاجين حل لمشكلة الـ **compatibility** بين السيرفر والـ apps اللي هرفعها عليه. ده غير **المشكلة التانية** اللي هتظهر نتيجة إني برفع كذا app على نفس السيرفر.

1.2 المشكلة التانية: كذا App على نفس السيرفر

قبل كده كانت الشركات بتحط كل الـ apps بتاعتها على نفس السيرفر، وكلهم بيشوفوا نفس الملفات وبيتشاركوا نفس الـ Resources بتاعة السيرفر. وهنا كان بيحصل عندنا شوية مشاكل:

- **استهلاك الموارد:** احتمال إن app واحد يستهلك موارد السيرفر بشكل كبير، فيأثّر على باقي الـ apps ويعطّلهم.
- **تعارض الـ Dependencies:** مثلاً app1 بيحتاج java بـ version معينة، و app2 بيحتاج java برضه بس بـ version مختلفة، وهكذا. فمع كترة الـ versions هيحصل عندي مشكلتين:
  1. استهلاك مساحة تخزين كبيرة.
  2. احتمال حدوث خطأ: مع كترة الـ apps، لو جيت أمسح app فيهم ونسيت هو كان شغال بأنهي version من java، ممكن أمسح version كانت بتشغّل app تاني.

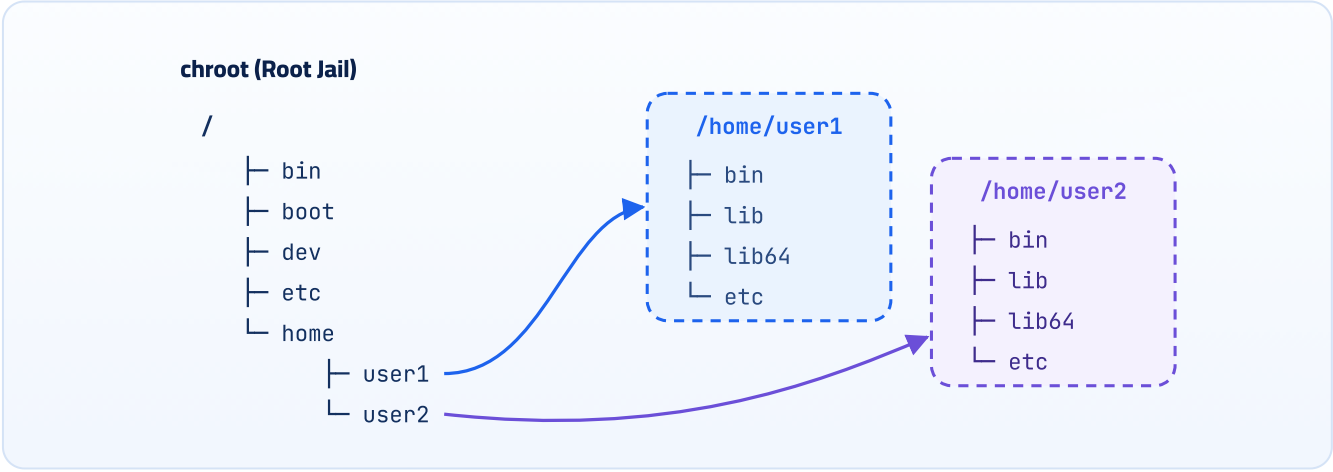


شكل 1.1 كل الـ apps على نفس السيرفر: موارد مشتركة، و versions مختلفة من نفس الـ dependency.

المشكلة دي اتطوّرلها كذا حل لمحاولة القضاء عليها، لأننا لقينا إننا محتاجين نعمل **عزل (Isolation)** بين كل app والتاني. هنتكلم عن الحلين اللي عندنا، بس الأول هنشرح الفكرة الأساسية اللي اتبنى عليها الحلين.

1.3 البداية: فكرة الـ chroot

ظهرت فكرة الـ **chroot**، وهي إني أعمل عزل لكل app عن التاني. يبقى كل app مش شايف ملفات النظام أو السيرفر نفسه، هو شايف root تاني خاص بيه مختلف عن root السيرفر، وشايف فولدرات زي `bin` و `lib` و `etc` خاصة بيه غير بتوع السيرفر، يعني كأنه على سيرفر لوحده. وبجانب ده اتعمل تحديد لاستهلاك كل app لموارد السيرفر باستخدام حاجة اسمها **ulimit**.



شكل 1.2 كل app شايف فولدر معين على إنه الـ root بتاعه (/)، وميعرفش حاجة عن باقي ملفات السيرفر.

> **معلومة مهمة**
>
> طبعاً الفكرة دي مكانتش كفاية إنها تبقى هي الحل الوحيد للمشكلة، بس كانت **الأساس** لحلول تانية بعد كده، زي اللي هنشوفهم دلوقتي.

1.4 الحل الأول: Virtual Machines

لو عندك سيرفر عليه Linux كنظام تشغيل رئيسي (Host OS)، وحبيت تقسّم السيرفر ده لأجزاء أصغر (Sub-Servers) كل جزء يشيل برنامج معين، كنت بتستخدم برنامج زي VirtualBox يعملك: VM 1 · VM 2 · VM 3.

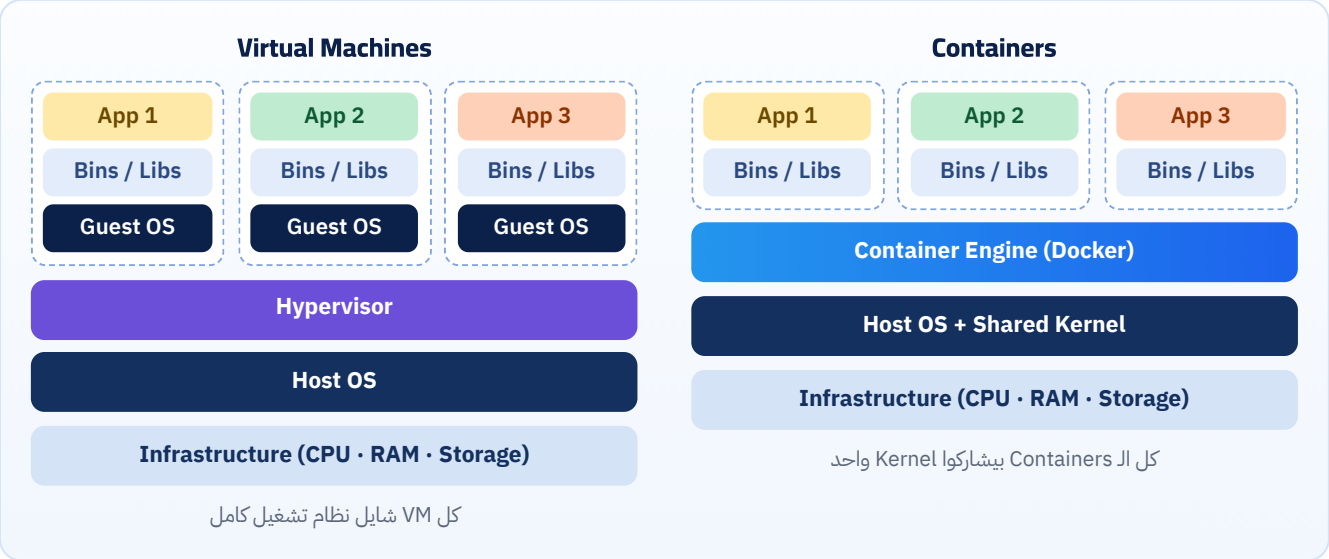
كل VM بيبقى كأنه سيرفر مستقل تماماً، وليه نظام تشغيل خاص بيه (Guest OS)، وبيتحط عليه الـ App وكل الـ dependencies الخاصة بيه، ونظام تشغيل كامل يقدر يستخدمه. الطبقات من تحت لفوق:

`Infrastructure`	الهاردوير نفسه: CPU و RAM و Storage
`Host OS`	نظام التشغيل الأساسي على الجهاز
`Hypervisor`	البرنامج اللي بيعمل الـ VMs ويديرها
`Guest OS`	نظام التشغيل اللي جوه كل VM

1.5 الحل التاني: Linux Containers

لو عندك سيرفر عليه Linux كنظام تشغيل أساسي (Host OS)، فبدل ما تعمل Virtual Machines وتنزّل Guest OS كامل لكل واحد، Linux وفّرلك طريقة أخف اسمها **OS-Level Virtualization**. يعني بدل ما نقسم السيرفر بأنظمة تشغيل كاملة، نقسمه على مستوى العمليات (Processes)، وهيبقى عندك حاجة اسمها **Container**.

كل Container بيبقى شايل app لوحده، وشغال كأنه process عادية في Linux، وكل الـ Containers بيشاركوا الـ **Kernel** بتاع الـ Host.



شكل 1.3 الفرق في المعمارية بين الـ Virtual Machines والـ Containers.

الـ Container بيعتمد على خاصيتين في Linux:

Linux Namespaces (1)

دي المسؤولة عن **العزل (Isolation)**. كل Container يحس إنه لوحده:

- ليه PID خاص
- ليه Network خاص
- ليه Filesystem view خاص
- ليه Hostname خاص

هو فاكر إنه لوحده على الجهاز، لكن فعلياً هو **Process معزولة**.

(cgroups) Control Groups (2)

دي المسؤولة عن **التحكم في الموارد**:

- CPU limit
- Memory limit
- I/O limit
- Network limit

يعني نقدر نقول: Container 1 ياخد 1GB RAM، و Container 2 ياخد 512MB RAM.

وجه المقارنة	Virtual Machine	Container
العزل بيتم على مستوى	الهاردوير (عن طريق Hypervisor)	نظام التشغيل (cgroups + Namespaces)
نظام التشغيل	Guest OS كامل جوه كل VM	مفيش Guest OS، بيستخدم Kernel الـ Host
الحجم	جيجات (GBs)	غالباً ميجات (MBs)
وقت التشغيل	دقايق (Boot كامل)	ثواني أو أقل (مجرد Process)
استهلاك الموارد	عالي: RAM و CPU لكل OS	خفيف جداً
العدد على نفس السيرفر	قليل	كبير جداً

قوة العزل	أقوى (Kernel منفصل)	كويس، بس الـ Kernel مشترك
أمثلة	VirtualBox و VMware و KVM	LXC و Docker و Podman

جدول 1.1 مقارنة سريعة بين الـ Virtual Machine والـ Container

الحلين دول كانوا كويسين، بس كل حل فيهم كان ليه عيوب، وواضح إن الأحسن فيهم فكرة الـ **Containers**، لأن مفيش Guest OS كامل، ومفيش Boot process مكرر، ومفيش Kernel مكرر، وكلهم بيستخدموا نفس الـ Kernel.

1.6 لكن الحلو مبيكملش: مشاكل الـ Linux Containers

زي ما احنا عارفين، الحلو مبيكملش. التعامل مع الـ Linux Containers بشكل مباشر كان فيه مشاكل عملية:

1. مفيش Standard واضح لبناء Container.
2. مفيش Distribution mechanism، يعني مفيش طريقة سهلة توزّع بيها الـ Container.
3. إدارة معقدة.
4. صعب على المطورين استخدامها.

كان الموضوع أقرب لإدارة نظام تشغيل، مش تجربة مطور سهلة.

من الآخر نقدر نقول إن Linux Containers وفّرت العزل والتحكم في الموارد، لكنها كانت تقنية منخفضة المستوى (Low-level). و **Docker** خد نفس التكنولوجيا وحوّلها لـ **Platform**:

جاهزة للإنتاج · سهلة الاستخدام · موحّدة المعايير · قابلة للتوزيع عالمياً

خلينا نرتب الصورة

- **المشكلة التانية** (كذا app على نفس السيرفر): حلّيناها بالـ Linux Containers اللي اتحسّن شكلها بـ Docker، وكده ضمنّا العزل والفصل بين كل app والتاني.
- **المشكلة الأولى** (الـ compatibility): حلها بقى موجود على طول بسبب Docker. محتجناش نطوّر حلول، Docker حلها بشكل مباشر، مش زي المشكلة التانية.

ملخص الفصل

- المشكلة الأولى: الـ app محتاج dependencies بـ versions معينة على السيرفر، ومن هنا جت "It works on my machine".
- المشكلة التانية: كذا app على نفس السيرفر بيتنافسوا على الموارد وبيتعارضوا في الـ versions، فمحتاجين عزل.
- الـ chroot كانت أول فكرة للعزل، وكانت الأساس للـ VMs والـ Containers.
- الـ VM فيها Guest OS كامل فوق Hypervisor، والـ Container مجرد Process معزولة بتشارك Kernel الـ Host.
- العزل بيتم بالـ Namespaces، والتحكم في الموارد بالـ cgroups.
- Docker خد فكرة الـ Linux Containers وخلاها Platform سهلة وموحّدة وقابلة للتوزيع.

أسئلة مراجعة

1. اشرح بكلامك معنى جملة "It works on my machine"، وليه كانت بتحصل؟
2. إيه المشكلتين اللي بيحصلوا لما نحط كذا app على نفس السيرفر؟
3. إيه الفرق الجوهري بين الـ VM والـ Container من ناحية نظام التشغيل؟
4. مين المسؤول عن العزل ومين المسؤول عن تحديد الموارد في الـ Linux Containers؟
5. ليه الـ Linux Containers لوحدها مكانتش كفاية، وإيه اللي Docker ضافه؟

02

الفصل التاني

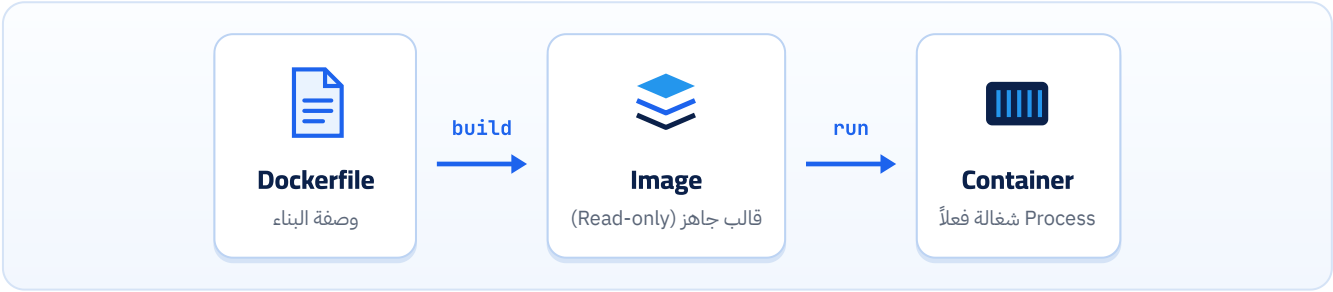
Docker: الفكرة وطريقة الشغل

Docker بيستخدم نفس فكرة الـ Linux Containers بس بشكل منظم: Dockerfile بيتعمله build فيطلع Image، والـ Image بنعملها run فيطلع Container. في الفصل ده هنفهم الرحلة دي كلها من غير ما نكتب ولا أمر.

Docker Containers
المشكلة الأولى اتحلّت إزاي؟
الرحلة من الـ Dockerfile لحد الـ Container
خطة الجزء العملي

2.1 Docker Containers

Docker بيعيد استخدام الـ Linux Containers ولكن بشكل أحسن، عن طريق إنه يبقى فيه ملف اسمه **Dockerfile** بيتكتب بمعايير أو syntax محددة، عشان لما يتعمله **build** يدينا حاجة اسمها **Image**. والـ Image دي بتكون أساس الـ Container، بمعنى إني لما بعمل **run** للـ Image بيتولد الـ **Container**، ويشتغل كـ Process عادية في النظام، بس بيحمل جواه الـ app اللي عايز تقوّمه على السيرفر.



شكل 2.1 من الـ Dockerfile للـ Image للـ Container.

Docker عمل 3 نقلات نوعية

Image جاهزة

بقى عندي Image جاهزة بتشتغل في أي مكان.

وصف واضح للبيئة

بقى فيه وصف واضح لبناء البيئة بفضل الـ Dockerfile.

توزيع عالمي

بقى فيه نظام توزيع عالمي، والـ images بتتنشر في كل مكان، وده بيسهّل ويسرّع رفع الـ apps على السيرفرات.

2.2 المشكلة الأولى اتحلّت إزاي؟

بالنسبة للمشكلة الأولى (الـ compatibility)، فبعد ما قررنا نستخدم Docker الوضع بيتغير تماماً عن الطريقة التقليدية. بدل ما كنا بنرفع الكود على السيرفر ونبدأ نسطّب عليه Node أو Java أو أي dependencies مطلوبة عشان المشروع يشتغل، دلوقتي أنا مجرد باخد **image** معمولة من **Dockerfile** فيه كل حاجة خاصة بالـ app ده، ولما أعمل run للـ image هتشغّل container على السيرفر، وده بيعادل تشغيل Process.

طب هل احتجت أسطّب حاجة على السيرفر عشان أقوّم الـ container اللي شايل الـ app؟

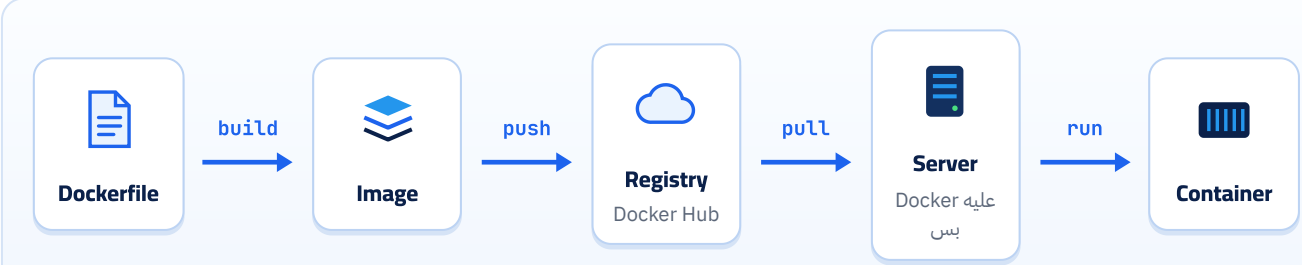
> **الإجابة: لأ**
>
> كل اللي احتجته على السيرفر إني سطّبت **Docker** بس. كل الـ dependencies والـ versions موجودة جوه الـ Image نفسها.

دي كانت مقدمة بس عشان نفهم الموضوع جه منين، وليه Docker جامد وبيطلّع نار من بُقّه.

2.3 الرحلة من الـ Dockerfile لحد الـ Container

عايزين نشوف الموضوع بيتم إزاي من أول ما نكتب الـ Dockerfile لحد ما نقوّم container على السيرفر. هنشوف الموضوع بشكل نظري الأول، وبعدها نشوفه بشكل عملي. دلوقتي كـ **DevOps Engineer** دي الخطوات اللي همشي عليها (ووارد أبدأ من خطوة متقدمة، إني أشتغل على image جاهزة وأنا مجرد بقوّم بيها الـ container وأتعامل معاها):

1. بكتب **Dockerfile** فيه مواصفات البيئة اللي التطبيق محتاجها، زي الـ Base Image اللي هبدأ منها (مثلاً `node:18-alpine`)، وبعدها بحدد مكان الشغل، وأنسخ الكود، وأنزّل الـ dependencies، وأحدد أمر التشغيل. فالـ Dockerfile ده يعتبر وصف لبناء Image.
2. بعد كده بعمل **build** للـ Dockerfile، سواء على جهازي أو عن طريق CI/CD (هنتكلم عنها بعدين)، فتتكوّن **Image** جاهزة فيها: الـ app نفسه + الـ runtime بتاعه + المكتبات اللي محتاجها + الـ filesystem الخاص بالتوزيعة اللي بدأت منها.
3. بعد ما الـ image تجهز، بترفعها على **Registry** زي Docker Hub.
4. على السيرفر اللي عليه Docker بس، كل اللي السيرفر بيعمله إنه يسحب الـ Image دي ويعملها **run**، فيشغّل منها **Container**.
5. ساعتها الـ app بيشتغل جوه بيئة معزولة، لكن باستخدام Kernel نظام التشغيل الأساسي للسيرفر.



شكل 2.2 دورة الشغل الكاملة كـ DevOps Engineer.

> **تشبيه يقرّب الصورة**
>
> تخيّل الموضوع كده: الـ **Dockerfile** هو كود الـ Mobile App. لما بنعمله build بيدّينا الـ **Image**، اللي هي نسخة الـ APK، اللي بعد كده بننقلها على أي تليفون عايزين نشغّل عليه الـ App ونعمله install ونشغّله. وده بالظبط إننا عملنا run للـ image وبقى عندنا **container** شغال.

2.4 خطة الجزء العملي

كل الكلام اللي فات ده ممكن لحد دلوقتي ميكونش واضح أوي، هيوضح أكتر لما ندخل للجزء العملي ونفهمه. الجزء العملي هيكون بالشكل ده:

01 **يعني إيه Images وإزاي بنتعامل معاها** الفصل 3	02 **عمل run لـ Images والتعامل مع الـ Containers** الفصل 4 و 5	03 **فهم Structure الـ Image، والـ Volumes، والـ Network** الفصل 6 و 7 و 8
04 **يعني إيه Dockerfile وإيه فايدته** الفصل 9	05 **إزاي نعمل build لـ Dockerfile ونطلّع Image وننشرها** الفصل 10 و 11 و 12	06 **يعني إيه Docker Compose وإيه فايدته** الفصل 13

طب ليه مبدأناش بالـ Dockerfile؟

ممكن تستغرب الترتيب ده، وليه مبدأناش بالـ Dockerfile رغم إنه المفروض أول حاجة. ببساطة لأن هدف Docker الفعلي مش إنك تبتدي من الصفر خالص، تطبيقاً لمبدأ **"متعيدش اختراع العجلة تاني"**. Docker بيوفّرلك images كتير جداً جاهزة (إما من الشركة نفسها أو من مطورين رافعين images على النت)، وانت تاخدها تشتغل بيها زي ما هي، أو تعدّل وتضيف عليها حاجات تخدم شغلك، وده بيكون عن طريق إنك تكتب Dockerfile.

فعلياً، انت بنسبة كبيرة جداً هتستخدم الـ Dockerfile لمجرد إنك تبني image جديدة من image قديمة، تاخدها وتعدّل وتضيف عليها عشان تعمل image جديدة تفيدك في شغلك. فعشان كده هنبتدي بإننا نفهم الـ images ونتعامل معاها كويس، وبعد كده نتعلم نبني بتاعتنا.

ملخص الفصل

- Dockerfile ← build ← Image ← run ← Container.
- الـ Image قالب جاهز فيه الـ app والـ runtime والمكتبات والـ filesystem.
- الـ Container هو Process معزولة شغالة من الـ Image.
- الـ Images بتترفع على Registry زي Docker Hub، والسيرفر بيعملها pull و run.
- السيرفر مش محتاج غير Docker بس، وده حل مشكلة الـ compatibility.

أسئلة مراجعة

1. رتّب الكلمات دي حسب دورة الشغل: Container، Dockerfile، Registry، Image.
2. إيه الحاجات اللي بتبقى جوه الـ Image بعد الـ build؟
3. ليه السيرفر مبقاش محتاج يتسطّب عليه Node أو Java؟
4. ليه الكتاب بدأ بالـ Images قبل الـ Dockerfile؟

الفصل التالت

الـ Images وإزاي بنتعامل معاها

قبل ما نبني Images بتاعتنا، لازم نعرف نتعامل مع الـ Images الجاهزة: نجيبها منين، نعرف اللي عندنا، ونمسح اللي مش محتاجينه.

يعني إيه Image؟ | بنجيب الـ Image منين؟ (docker pull) | أعرف إيه الـ Images اللي نازلة عندي؟ | حذف Image

3.1 يعني إيه Image؟

- زي ما فهمنا قبل كده، الـ Container زي VM شغال على الجهاز أو السيرفر، الفرق إنه شغال في هيئة process عادية. بس بتقدر تدخل عليه وهو شغال، تفتح shell وتكتب أوامر، وجواه ملفات تقدر تشوفها وتتعامل معاها، وتعمل run لـ app وهكذا.
- فبالتالي انت محتاج مواصفات وحاجات محددة خاصة بالـ environment دي، والـ **Image** هي اللي بيكون فيها الكلام ده، بس بتكون زي **template** مبني مسبقاً، بتعمله run يقوّملك container على طول.
- ممكن تتخيلها زي **قالب** لمجسّم، بتصب فيه مادة تتحول للمجسّم ده. تاخده تستخدمه وتنقله وتشاركه وتشغّله عند أي حد، وهيشتغل على طول ومش هيحصل مشاكل اختلاف الـ versions وغيرها.

> **على الماك: Docker شغال جوه Linux صغير**
>
> قلنا إن الـ Containers بتشارك الـ **Kernel** (قلب نظام التشغيل) بتاع Linux. طب الماك مش Linux! فـ **Docker Desktop** (أو OrbStack أو Colima) بيعمل VM صغيرة جداً فيها Linux من غير ما تحس، والـ Containers بتشتغل جواها. وده معناه كام حاجة:
>
> - سطّبه بـ `brew install --cask docker` وافتح Docker Desktop مرة، أو حمّله من الرابط تحت. لازم يكون شغال (الحوت فوق في الشريط) قبل أي أمر `docker`.
> - كل أوامر الكتاب زي ما هي بالظبط، و `-p 8080:80` و `localhost:8080` شغالين عادي.
> - المسار `/var/lib/docker/...` اللي هتشوفه في فصل الـ Volumes موجود جوه الـ VM مش على الماك، فمش هتلاقيه في الـ Finder.
> - اختار Images بتشتغل على معالجات Apple (`arm64`). الرسمية كلها بتدعمها.

> **قبل أي حاجة: سطّب Docker**
>
> لازم نسطّب Docker على الجهاز من الرابط ده: docs.docker.com/get-started/get-docker
>
> وبعد التسطيب اتأكد إنه شغال بالأمرين دول: `docker --version` و `docker run hello-world`.

3.2 بنجيب الـ Image منين؟ (docker pull)

بنسحب الـ image من أي **Docker Registry** عليه images، زي **Docker Hub** (وفيه غيره عادي).

```
Terminal
docker pull [REGISTRY/REPOSITORY/]image_name[:TAG]
```

معناها: هات Image اسمها `image_name` بالـ version بتاعها `TAG`، من الـ Docker Registry اللي اسمها `REGISTRY`، الموجودة في Repository اسمها `REPOSITORY`، ونزّلها على جهازي.



شكل 3.1 تشريح الاسم الكامل لأي Image.

```
Terminal
docker pull busybox
```

يعني: روح هات الـ Image اللي اسمها `busybox`، أحدث version منها، من Docker Hub. هيشوف لو موجودة عندي ممكن ميعملش حاجة، ولو مش موجودة يبدأ يسحبها.

القيم الافتراضية

- لو مكتبتش `:TAG` ← هياخد `latest` (أحدث version).
- لو مكتبتش `REGISTRY/REPOSITORY/` ← هيروح على Docker Hub: `docker.io/library/`

> **خلي بالك**
>
> لو هتسحب من موقع تاني، أو من Docker Hub برضه بس من **REPOSITORY خاصة**، فالموضوع بيكون مختلف، وهتحتاج تزوّد كام خطوة قبلها (زي `docker login`)، وهنشرحهم في فصل رفع الـ Images على Docker Hub.

3.3 أعرف إيه الـ Images اللي نازلة عندي؟

```
Terminal
docker images
```

هيوريك بيانات عن كل Image عندك: **REPOSITORY** (اسم الـ Image)، و **TAG**، و **IMAGE ID**، و **SIZE**. مثلاً:

```
Terminal
moaz@Linux:~$ docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
nginx        latest    a72860cb95fd   2 weeks ago   188MB
busybox      latest    6fd955f66c23   1 month ago   4.26MB
```

3.4 حذف Image

`docker rmi image-name`	بيحذف الـ Image. مش هتتحذف لو فيه Container بيستخدمها.
`docker rmi -f image-name`	بيحذفها بالإجبار. لكن ده خطر لو حد شغال عليها.

Best Practice

قبل ما تمسح Image، اعرف مين بيستخدمها بـ `docker ps -a` ، وامسح الـ Containers القديمة الأول بدل ما تستخدم `-f` .

ملخص الفصل

- الـ Image قالب جاهز (template) بيتعمله run فيطلع Container.
- `docker pull` بيسحب Image من Registry، والافتراضي `docker.io/library` و `latest` .
- `docker images` بيعرض الـ Images اللي عندك.
- `docker rmi` بيمسح Image، و `-f` بالإجبار (بحذر).

أسئلة مراجعة

1. اكتب الاسم الكامل اللي Docker بيفهمه لما تكتب `docker pull nginx` .
2. إيه اللي هيحصل لو عملت pull لـ Image موجودة عندك فعلاً؟
3. ليه `docker rmi` ممكن يرفض يمسح Image؟

الفصل الرابع

04 تشغيل الـ Images والتعامل مع الـ Containers

هنا بنبدأ الشغل الحقيقي: نعمل run لـ Image، نفهم ليه الـ Container بيقف لوحده، نشغّله في الخلفية، ندخل جواه، ونفتح بورت عشان نشوف موقع nginx من المتصفح.

تشغيل Container (docker run) | ليه الـ Container اشتغل وخلص على طول؟ | عرض الـ Containers (docker ps)
أوامر التعامل مع الـ Container | مثال عملي: busybox | مثال عملي: nginx Web Server | خصائص الـ Container في سطور

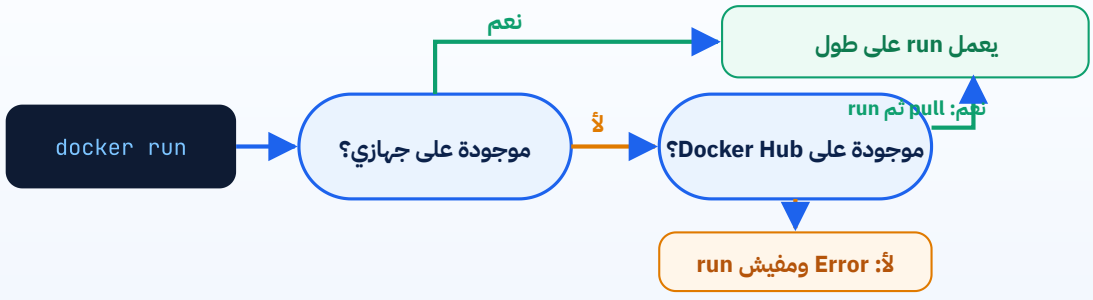
4.1 تشغيل Container (docker run)

Terminal

```
docker run [--name container_name] image_name
```

معناها: اعمل run للـ Image اللي اسمها `image_name`، وشغّلها كـ Container عندي على الجهاز.

- لو الـ image موجودة على جهازي بالفعل، هيعملها run عادي.
- أما لو مش موجودة، هيشوف لو هي موجودة على Docker Hub تحديداً على `docker.io/library/`:
  - لو لقاها، هيروح من نفسه يسحبها وينزّلها على الجهاز وبعدها يعملها run.
  - لو ملقاهاش، مش هيعمل run أصلاً.



شكل 4.1 Docker بيدوّر على الـ Image فين قبل ما يعمل run.

Best Practice

لو مكتبتش `--name container_name`، Docker هيدّي الـ container اسم عشوائي (زي `keen_jang`). الأحسن تسمّيه بنفسك بدل ما تستخدم الـ ID بتاعه، لأنه بيكون طويل، فاسم واضح أحسن بكتير.

4.2 ليه الـ Container اشتغل وخلص على طول؟

بعد ما عملنا run للـ image هينشأ Container عندنا. هتلاحظ إن الـ Container اللي انت لسه مقوّمه اشتغل وخلص على طول، وكمان إنه لما اشتغل خد الـ Terminal ومبقاش متاح ليك تستخدمه، ولما خلص رجّعهولك تاني.

وده منطقي جداً، لأن ببساطة الـ Container في الأول والآخر **Process** بتشتغل، ولما بتخلص بتقف. نفس الكلام لما بتقوّم Container: طالما مفيش حاجة تجبره يفضل شغال، هو بيقوم وبعدها يخلص ويقف.

عشان يفضل شغال

تدّيله أمر يجبره على كده، زي إنك تكتب `sleep time` في آخر السطر، فكده انت بتقوله يفضل شغال لمدة `time` ثانية. ولو عايزه يفضل شغال على طول اكتب `sleep infinity`.

عشان ميخدش منك الـ Terminal

اكتب `-d` بعد `run` عشان تفعّل الـ **Detached Mode**، يعني التشغيل في الخلفية.

Terminal

```
docker run -d busybox sleep infinity
```

4.3 عرض الـ Containers (docker ps)

`docker ps`	بيعرض الـ Containers الشغالة بس (running).
`docker ps -a`	بيعرض كل الـ Containers: الشغالة واللي وقفت (running و ended).

العمود	معناه
CONTAINER ID	رقم تعريف الكونتينر
IMAGE	الـ image اللي اتعمل منها الكونتينر
COMMAND	الأمر اللي شغال جوه الكونتينر
CREATED	وقت إنشاء الكونتينر
STATUS	حالته (Up يعني شغال)
PORTS	البورتات المفتوحة
NAMES	اسم الكونتينر

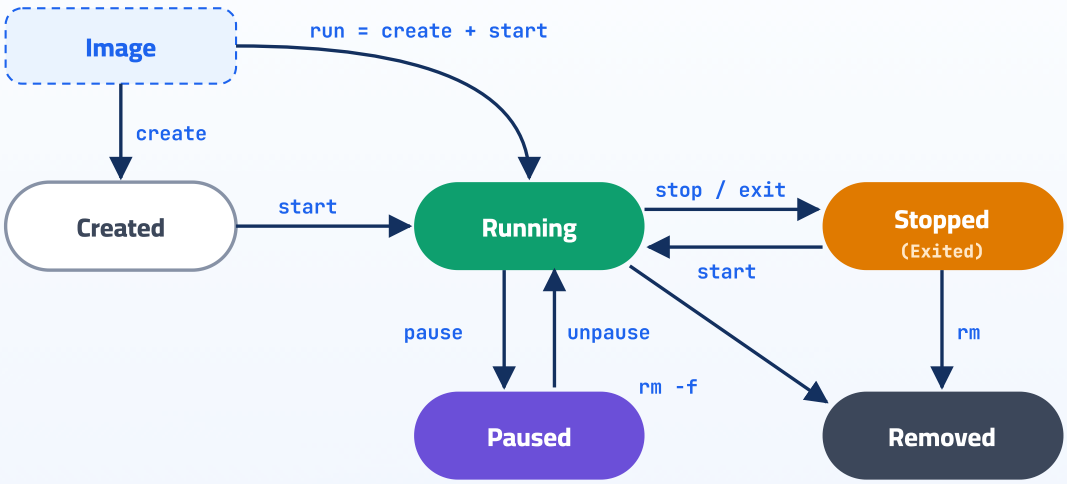
جدول 4.1 معنى كل عمود في ناتج `docker ps`

Container = Process

لو لاحظت، هتلاقيك كتبت أمر `ps` اللي كنا بنكتبه لما نحب نشوف الـ Processes الشغالة على الجهاز. وده يفسّر ليه بنقول إن **الـ Process = Container**، فعشان كده لما حبينا نشوف الـ Containers الشغالة كتبنا `ps` ، بس قبلها `docker` عشان نوضّح إننا عايزين نشوف الـ Docker Containers.

4.4 أوامر التعامل مع الـ Container

`docker exec -it CONTAINER_ID sh`	هيدخّلك جوه الـ Container ويفتحلك shell، تقدر تكتب commands وتتعامل مع الملفات اللي جواه (كأنك جوه VM عادي جداً).
`docker stop CONTAINER_ID`	هيوقّف الـ Container لو هو شغال.
`docker start CONTAINER_ID`	هيشغّل الـ Container لو كان واقف بعد ما كان شغال.
`docker rm CONTAINER_ID`	هيحذف الـ Container. لازم تبقى حذر في الخطوة دي، خصوصاً لو فيه Container تاني معتمد عليه.
`docker rm -f CONTAINER_ID`	حذف Container بس بالإجبار (حتى لو شغال).



شكل 4.2 دورة حياة الـ Container والأوامر اللي بتنقله من حالة للتانية.

4.5 مثال عملي: busybox

1. نشغّل Container من الـ image اللي اسمها busybox، ونخليه شغال في الخلفية (`-d`) وشغال على طول (`sleep infinity`):

```
Terminal
moaz@Linux:~$ docker run -d busybox sleep infinity
b4fcc648e91d464a41922f14e598f7eb9f1f9e881c91a5ee0fa435c828a4db23
```

2 نشوف الـ Containers الشغالة عشان نعرف الـ ID بتاع الـ Container اللي لسه مشغّلينه:

```
Terminal

moaz@Linux:~$ docker ps
CONTAINER ID   IMAGE     COMMAND            CREATED         STATUS         PORTS   NAMES
b4fcc648e91d   busybox   "sleep infinity"   2 minutes ago   Up 2 minutes           keen_jang
```

3 ندخل على الـ Container اللي لسه مشغّلينه بالـ ID بتاعه، عشان نجرّب نكتب أي أوامر أو نتعامل مع الملفات اللي فيه:

```
Terminal

moaz@Linux:~$ docker exec -it b4fcc648e91d sh
/ #
```

4 نجرّب نكتب `ls` مثلاً، هنلاقي إننا شايفين الملفات والفولدرات اللي جوه الـ Container، وهنلاقيها شبه الفولدرات اللي في أي VM، كأننا مقوّمين جهاز بالفعل:

```
Terminal

/ # ls
bin   dev   etc   home   lib   lib64   proc   root   sys   tmp   usr   var
```

5 نجرّب مثلاً نعمل ملف جديد اسمه `file.txt`، هنلاقي إننا شايفين الملفات والفولدرات اللي كانت موجودة جوه الـ Container من قبل، والملف اللي لسه عاملينه:

```
Terminal

/ # touch file.txt
/ # ls
bin   dev   etc   file.txt   home   lib   lib64   proc   root   sys   tmp   usr   var
```

6 هنلاحظ حاجة مهمة: لما نقفل الـ Container ده خالص (نمسحه) ونقوّم Container جديد من نفس الـ image، مش هنلاقي الملف اللي لسه عاملينه:

```
Terminal

/ # ls
bin   dev   etc   home   lib   lib64   proc   root   sys   tmp   usr   var
```

ده راجع لسبب معين في هيكل الـ image نفسها، هنشرحه بالتفصيل في فصل هيكل الـ Image.

> **⚠ خلي بالك**
>
> الفرق بين `stop` و `rm` مهم: لو عملت `docker stop` وبعدين `docker start` لنفس الـ Container، الملف هيفضل موجود. الملف بيضيع لما الـ Container **يتمسح** (`docker rm`) ونشغّل واحد جديد من الـ image.

4.6 مثال عملي: nginx Web Server

1 نشغّل Container من الـ image اللي اسمها nginx ونخليه شغال في الخلفية (`-d`). الـ Image دي فيها أمر تشغيل افتراضي، فمش محتاج تكتب `sleep infinity` عشان تجبره يفضل شغال:

```
Terminal
docker run -d nginx
```

كده أنا أقدر أجرّب nginx من غير ما أسطّبه عندي على الجهاز.

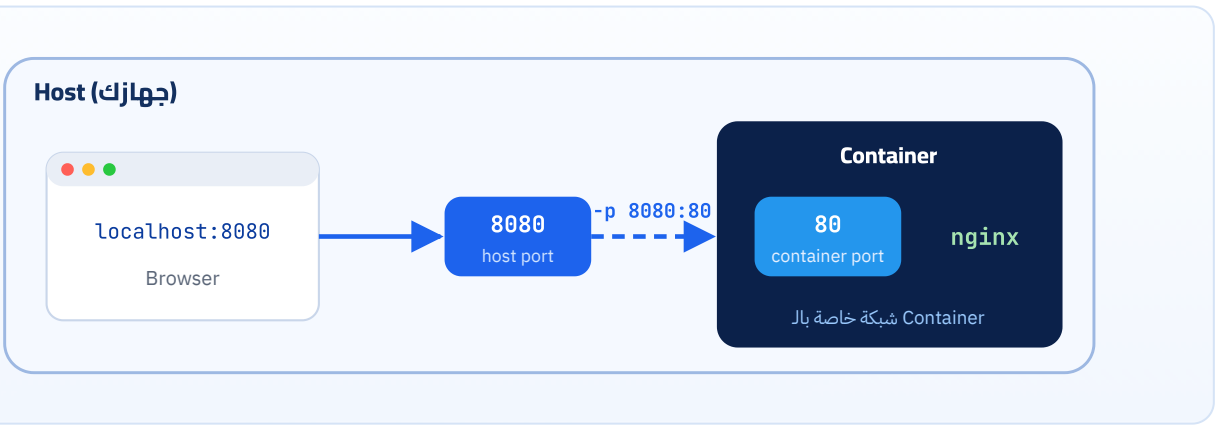
2 لكن هلاحظ مشكلة: لما بجرّب أدخل على `http://localhost:8080` هلاقي إن صفحة nginx الافتراضية مش موجودة، والمتصفح مطلّعلي رسالة **"This site can't be reached"**.

السبب إن الـ nginx شغال جوه الـ Container ده بس، فكأنه شغال على جهاز منفصل مع نفسه. فعشان نقدر نخليه يسمع على جهازنا (الـ host)، لازم نفهم إنه بما إن الـ Container يعتبر جهاز خاص، فأكيد ليه شبكة خاصة بيه، و nginx بيسمع على port معين جوه الـ Container (اللي هو 80).

3 هنا هنضطر نقول للـ Container إنه يوصّل الـ Port اللي عنده (جواه) بـ Port عندنا على الـ Host، والعملية دي اسمها **Port Mapping**. بنعملها بطريقتين:

`docker run -d -P nginx`	يعني: افتح Port عشوائي عندي (أيًّا كان رقمه) ووصّله بـ 80 جوه الـ Container.
`docker run -d -p 8080:80 nginx`	يعني: افتح Port 8080 عندي ووصّله بـ 80 جوه الـ Container.

4 ساعتها لو دخلت على `http://localhost:8080` هتشوف الصفحة دي:



شكل 4.3 الـ Port Mapping: الطلب بيوصل لبورت 8080 على الـ Host، و Docker بيحوّله لبورت 80 جوه الـ Container.



شكل 4.4 صفحة nginx الافتراضية بعد ما عملنا Port Mapping صح.

الصيغة	بتعمل إيه	إمتى تستخدمها
`-p 8080:80`	انت اللي بتحدد رقم البورت على الـ Host	أغلب الوقت، علشان تعرف الرابط ثابت
`-P`	Docker بيختار بورت عشوائي لكل بورت معمول له EXPOSE	للتجربة السريعة (اعرف الرقم بـ `docker ps`)

جدول 4.2 الفرق بين `-p` و `-P`

4.7 خصائص الـ Container في سطور

عايزين نقول كام ملحوظة عن الـ Container ونلخّص أهم النقط:

بيستخدم نفس Kernel السيرفر

- مفيش Guest OS.
- مفيش Kernel خاص بيه، بيشارك الـ Host Kernel.
- علشان كده أخف وأسرع من الـ VM.

ليه ملفات و Packages خاصة بيه

- filesystem خاص (Layer معزولة).
- Libraries و Dependencies خاصة.
- Environment variables خاصة.

يعني Container A ممكن يستخدم Python 3.8، و Container B يستخدم Python 3.12، بدون تعارض.

هو في الأساس Process

الـ Container مش "جهاز"، هو مجموعة Processes معزولة جوه النظام. لو عملت `ps aux` على السيرفر، هتلاقي عمليات الكونتينر ضمن عمليات السيرفر.

بيبدأ ويقف بسرعة جداً

لأنه مفيش Boot لنظام تشغيل، ومفيش BIOS، ومفيش Kernel load. هو مجرد تشغيل Process.

خفيف في استهلاك الموارد

لأنه مفيش RAM لنظام تشغيل كامل، ولا CPU overhead للـ Hypervisor، فتقدر تشغّل عدد أكبر مقارنة بالـ VMs.

Stateless افتراضياً

بمعنى إن أي بيانات جوه الـ writable layer هتضيع لو الكونتينر اتمسح (وده اللي هنحله بالـ Volumes).

بيخليك تجرّب من غير ما تسطّب

تقدر تنزّل image جاهزة فيها packages عايز تجربها من غير ما تسطّبها على السيرفر وتاخد مساحة. مثلاً busybox جواها أوامر مش بتشتغل عندك على Linux إلا لو مسطّب packages معينة، فتعمل run للـ image وتجرّبها من جوه الـ Container.

قابل للنقل (Portable)

لو عندك Image، هتشتغل على أي سيرفر فيه Docker بنفس السلوك، ونفس البيئة، ونفس الإعدادات. وجملة "It works on my machine" تنتهي هنا.

> ⚠ **خلي بالك: الـ Kernel مشترك**
>
> لو السيرفر Linux، الكونتينر لازم يكون Linux. مينفعش تشغّل Windows container على Linux مباشرة.

ملخص الفصل

- `docker run` بيشغّل Container من Image، ولو مش موجودة بيعملها pull الأول.
- الـ Container بيقف لما الـ Process الأساسية تخلص، و `-d` بيشغّله في الخلفية.
- `docker ps` للشغال، و `docker ps -a` للكل.
- `exec -it ... sh` للدخول، و `stop` و `start` و `rm` للإدارة.
- علشان توصل لخدمة جوه الـ Container من جهازك لازم Port Mapping بـ `-p host:container`.
- الملفات اللي بتتعمل جوه الـ Container بتضيع لما الـ Container يتمسح.

أسئلة مراجعة

1. ليه `docker run busybox` بيخلص على طول، وإزاي تخليه يفضل شغال؟
2. إيه الفرق بين `docker ps` و `docker ps -a`؟
3. اكتب الأمر اللي يشغّل nginx في الخلفية باسم `web` على بورت 3000 في جهازك.
4. عملت ملف جوه Container وبعدين `stop` ثم `start`، الملف موجود ولا لأ؟ وليه؟
5. ليه مينفعش نشغّل Windows container على سيرفر Linux؟

الفصل الخامس

05

مراقبة وفحص الـ Containers: logs و inspect و stats

الـ Container شغال في الخلفية، طب أعرف منين هو بيعمل إيه؟ بيطبع إيه؟ واخد IP كام؟ وبياكل RAM قد إيه؟ التلات أوامر دول هما عينك على أي Container.

- docker logs: الـ Container بيقول إيه؟
- docker inspect: كل التفاصيل عن أي حاجة
- docker stats: بياكل موارد قد إيه؟
- أوامر مساعدة مفيدة

إضافة

الفصل ده **إضافة** على الشرح الأصلي علشان الكتاب يبقى كامل، ومكتوب بنفس الأسلوب.

5.1 docker logs: الـ Container بيقول إيه؟

لما بتشغّل Container بـ `-d` مش بتشوف أي حاجة بيطبعها. الـ Process الأساسية جوه الـ Container (اللي اتشغلت بالـ CMD أو الـ ENTRYPOINT) أي حاجة بتطبعها على الـ **stdout** أو الـ **stderr**، Docker بيحفظها، وتقدر تشوفها بـ:

`docker logs web`	يعرض كل الـ logs بتاعة الـ Container اللي اسمه `web`.
`docker logs -f web`	يفضل متابع (follow) ويعرض أي log جديد أول بأول، زي `tail -f`. اخرج بـ `Ctrl+C`.
`docker logs --tail 50 web`	آخر 50 سطر بس.
`docker logs --since 10m -t web`	الـ logs بتاعة آخر 10 دقايق، ومعاها الوقت (timestamps).

مثال عملي

شغّل nginx وافتح الصفحة من المتصفح كام مرة، وبعدين:

Terminal

```
moaz@Linux:~$ docker run -d --name web -p 8080:80 nginx
moaz@Linux:~$ docker logs --tail 2 web
172.17.0.1 - - [22/Oct/2026:11:02:13 +0000] "GET / HTTP/1.1" 200 615 "-" "Mozilla/5.0
..."
172.17.0.1 - - [22/Oct/2026:11:02:14 +0000] "GET /favicon.ico HTTP/1.1" 404 153 "-"
"Mozilla/5.0 ..."
```

كل request وصل لـ nginx ظهر في الـ logs.

Best Practice

خلي التطبيق بتاعك يكتب الـ logs على الـ stdout والـ stderr بدل ملفات جوه الـ Container. كده `docker logs` يشوفها، وأي أداة مراقبة تقدر تجمعها، ومش هتضيع لما الـ Container يتمسح لو بعتّها برّه.

5.2 docker inspect: كل التفاصيل عن أي حاجة

`docker inspect` بيطلّعلك ملف **JSON** فيه كل حاجة Docker عارفها عن الـ Container (أو الـ Image أو الـ Volume أو الـ Network): حالته، والـ IP بتاعه، والـ Ports، والـ Volumes المتوصّلة بيه، والـ Environment variables، والأمر اللي شغّال بيه.

Terminal

```
docker inspect web
```

الناتج طويل جداً، فبنستخدم `--format` (أو `-f`) عشان نطلّع المعلومة اللي محتاجينها بس:

`docker inspect -f '{{.State.Status}}' web`	حالة الـ Container (running أو exited أو ...).
`docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' web`	الـ IP بتاع الـ Container جوه شبكة Docker.
`docker inspect -f '{{json .Mounts}}' web`	الـ Volumes والـ Mounts المتوصّلة بيه.
`docker image inspect nginx`	تفاصيل الـ Image نفسها: الـ Layers والـ CMD والـ ENV والـ Ports.

5.3 docker stats: بياكل موارد قد إيه؟

فاكر الـ **cgroups** اللي اتكلمنا عنها في المقدمة؟ `docker stats` بيوريك استهلاك كل Container للموارد بشكل لحظي: الـ CPU، والـ RAM، والـ Network، والـ Disk I/O.

Terminal

```
moaz@Linux:~$ docker stats --no-stream
CONTAINER ID   NAME   CPU %   MEM USAGE / LIMIT     MEM %   NET I/O         BLOCK I/O   PIDS
3f2a9c1d7e44   web    0.00%   12.4MiB / 512MiB      2.42%   4.1kB / 2.3kB   0B / 4.1kB  9
```

من غير `--no-stream` الأمر هيفضل يحدّث الأرقام كل ثانية لحد ما تدوس `Ctrl+C`. وتقدر تحدد حدود للموارد وانت بتشغّل الـ Container، وده بالظبط شغل الـ cgroups:

Terminal

```
docker run -d --name web --memory 512m --cpus 1 -p 8080:80 nginx
```

الأمر	بيجاوب على سؤال	مثال استخدام
`docker logs`	الـ app بيقول إيه؟ فيه Errors؟	الـ Container بيقع على طول ومش عارف ليه
`docker inspect`	الـ Container متظبط إزاي؟	عايز تعرف الـ IP أو الـ Volume متوصّل فين
`docker stats`	بياكل موارد قد إيه دلوقتي؟	السيرفر تقيل وعايز تعرف مين السبب

جدول 5.1 التلات أوامر: كل واحد بيجاوب على سؤال

5.4 أوامر مساعدة مفيدة

`docker top web`	الـ Processes اللي شغالة جوه الـ Container.
`docker port web`	الـ Port Mapping بتاع الـ Container.
`docker system df`	المساحة اللي واخدينها الـ Images والـ Containers والـ Volumes.
`docker container prune`	يمسح كل الـ Containers الواقفة. اتأكد إنك مش محتاج حاجة منهم قبل ما توافق.

خلي بالك

لو الـ Container بيقع على طول بعد ما يقوم (STATUS بتاعه `Exited (1)`)، أول حاجة تعملها `docker logs` عليه، حتى لو هو واقف. الـ logs بتفضل موجودة لحد ما الـ Container يتمسح.

ملخص الفصل

- `docker logs` بيعرض الـ stdout والـ stderr بتوع الـ Process الأساسية، و `-f` للمتابعة.
- `docker inspect` بيعرض كل التفاصيل في JSON، و `-f` لاستخراج قيمة واحدة.
- `docker stats` بيعرض استهلاك الموارد لحظياً، و `--memory` و `--cpus` لتحديدها.

أسئلة مراجعة

1. Container بيقع بعد ما يقوم بثانية، إيه أول أمر هتكتبه؟
2. إزاي تعرف الـ IP بتاع Container من غير ما تقرا الـ JSON كله؟
3. اكتب أمر يشغّل Container ميعديش 256 ميجا RAM.

الفصل السادس

فهم الـ Image: الـ Layers والـ Structure

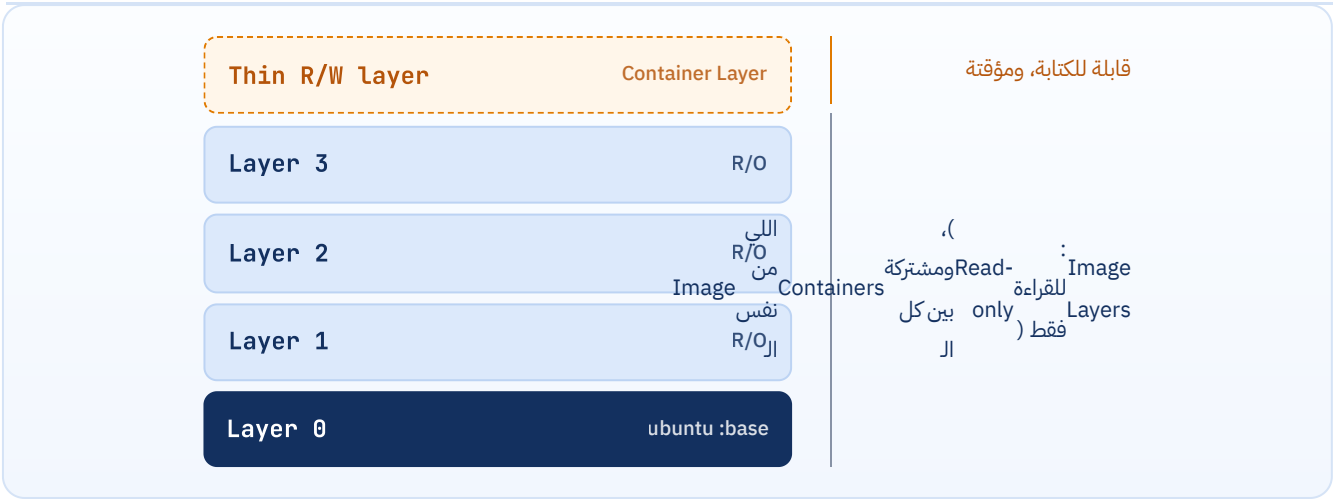
ليه الملف اللي عملناه جوه الـ Container اختفى؟ الإجابة في طريقة بناء الـ Image نفسها: طبقات فوق بعض، كلها Read-only، وفوقها طبقة واحدة بس قابلة للكتابة.

الـ Image مكوّنة من Layers | الـ Container Layer مؤقتة (Volatile) | لو عملت ملف جديد جوه الـ Container؟
لو عدّلت ملف موجود؟ (Copy-on-Write) | لو مسحت ملف موجود؟ (Whiteout)

6.1 الـ Image مكوّنة من Layers

بما إننا عرفنا قبل كده إن الـ Container بيتعمل أصلاً من الـ Image، فده معناه إن أي حاجة إحنا بنشوفها جوه الـ Container، سواء ملفات أو برامج أو إعدادات أو أي behavior، كلها جاية أساساً من الـ Image. فلو لاحظنا سلوك معين في الـ Container، زي مثلاً إن التعديلات بتختفي لما الكونتينر يتمسح، فده معناه إن السبب الحقيقي غالباً راجع لطريقة تصميم الـ Image نفسها. علشان كده لازم نفهم الـ Image شغالة إزاي من جوه.

- الحقيقة إن الـ Image في Docker مش عبارة عن حاجة واحدة بس، لكنها بتتكوّن من مجموعة طبقات اسمها **Layers**.
- الـ Layers دي هي اللي بتكوّن محتوى الـ Image بالكامل، وهي اللي بتحدد الملفات والبرامج وكل حاجة إحنا بنشوفها لما نشغّل Container منها.
- الطبقات دي بتكون **Immutable**، يعني غير قابلة للتعديل: تقدر تشوف محتواها وتستخدمه، لكن ما ينفعش تعدّل فيها بشكل مباشر. ولو حاولت تعمل تعديل (زي تعديل ملف أو إضافة ملف جديد)، Docker ما بيغيّرش في الـ Layers الأصلية، لكنه بيعمل Layer جديدة فوقها ويحط فيها التعديل اللي حصل.
- لما بنشغّل Container من Image، Docker بيجمّع كل الـ Layers بتاعة الـ Image فوق بعضها علشان يبني منها نظام الملفات اللي إحنا بنشوفه جوه الـ Container. والطريقة دي اسمها **Union File System**، وفكرتها ببساطة إن فيه كذا Layer فوق بعض، لكن إحنا شايفينهم كأنهم File System واحد.
- بعد كده لما الـ Container يشتغل، أي تعديل أو إضافة بتحصل أثناء التشغيل بتتحط في Layer جديدة فوقهم اسمها **Container Layer**، والـ Layer دي هي الوحيدة اللي بتكون قابلة للتعديل.



شكل 6.1 الـ Image عبارة عن Layers للقراءة فقط، والـ Container بيضيف فوقها طبقة رقيقة قابلة للكتابة.

> **معلومة إضافية: شوف الـ Layers بعينك**
>
> الأمر `docker image history nginx` بيعرضلك كل Layer في الـ Image، وحجمها، والخطوة اللي عملتها.

6.2 الـ Container Layer مؤقتة (Volatile)

- المهم نفهم إن الـ Container Layer دي مؤقتة أو **Volatile**، يعني زي الرامات في الكمبيوتر بالظبط: أول ما البرنامج يقفل، البيانات اللي فيها بتختفي.
- بنفس الفكرة، أول ما الـ Container يتشال أو يتمسح، الـ Layer دي بتروح بكل اللي فيها.
- عشان كده لما تشغّل Container جديد من نفس الـ Image، كل حاجة بترجع زي ما كانت.

6.3 لو عملت ملف جديد جوه الـ Container؟

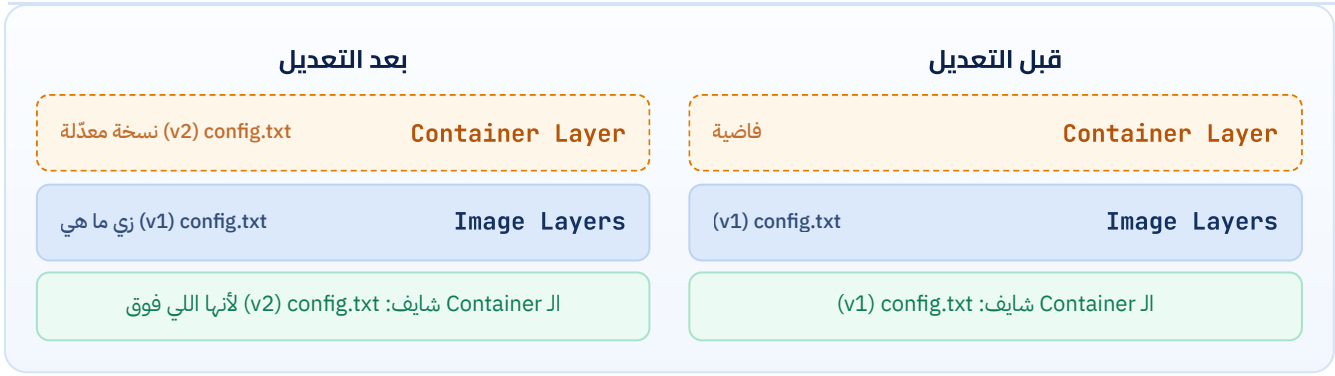
Docker بيحط الملف اللي اتضاف في الـ **Container Layer** اللي فوق، والـ Image Layers نفسها مبتتغيرش.

6.4 لو عدّلت ملف موجود؟ (Copy-on-Write)

Docker ما يقدرش يعدّل الملف في مكانه، لأن الطبقات الأصلية Immutable. فاللي بيحصل إنه بياخد نسخة من الملف ده ويحطها في الـ Container Layer، وبعد كده يعمل التعديل على النسخة دي. فكده يبقى عندك:

- نسخة أصلية موجودة في الـ Layers بتاعة الـ Image.
- نسخة معدّلة موجودة في الـ Container Layer.

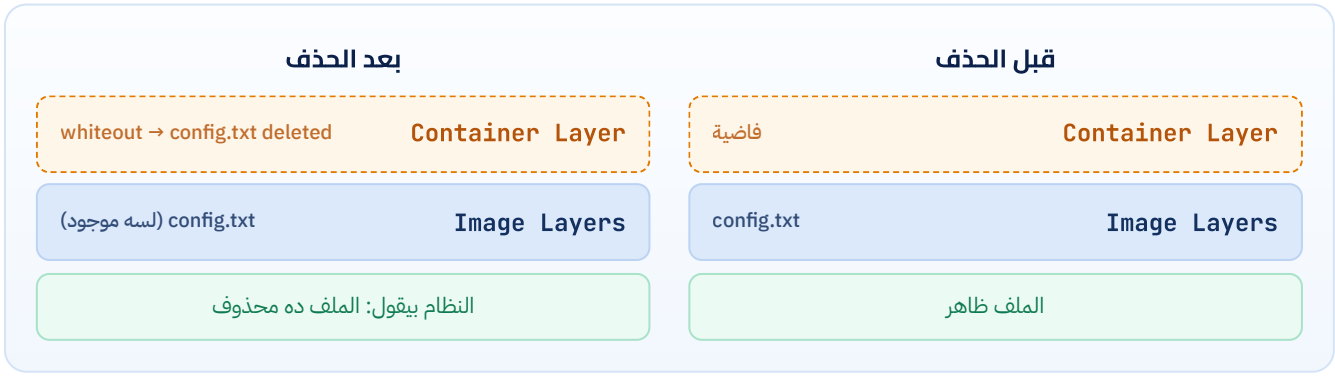
والسيستم دايماً بيشوف النسخة اللي فوق. ولو مسحت الـ Container، الـ Container Layer بتختفي، ويرجع الملف يظهر زي ما كان في الـ Image.



شكل 6.2 الـ Copy-on-Write: التعديل بيتعمل على نسخة في الـ Container Layer، والأصل مبيتلمسش.

6.5 لو مسحت ملف موجود؟ (Whiteout)

لو عندنا ملف اسمه `/etc/config.txt` ومسحناه، Docker ما يمسحش الملف الحقيقي. بدل كده بيعمل حاجة اسمها **Whiteout file**، أو سجل بيقول إن الملف ده محذوف، والسجل ده بيتحط في الـ Container Layer.



شكل 6.3 الـ Whiteout: الملف لسه موجود فعلياً في الـ Image، بس مستخبي.

> **خلي بالك**
>
> يعني مسح ملف كبير جوه الـ Container (أو في سطر RUN لاحق في الـ Dockerfile) **مش بيصغّر** حجم الـ Image، لأن الملف لسه موجود في الـ Layer اللي تحت. وده هيفرق معانا جداً لما نكتب Dockerfile.

ملخص الفصل

- الـ Image مجموعة Layers للقراءة فقط (Immutable) فوق بعض.
- الـ Union File System بيخلّي الطبقات تبان كأنها File System واحد.
- كل Container بياخد Container Layer رقيقة قابلة للكتابة، وبتضيع لما الـ Container يتمسح.
- التعديل = Copy-on-Write، والحذف = Whiteout، والأصل في الـ Image مبيتغيرش.

؟ أسئلة مراجعة

1. ليه تقدر تشغّل 10 Containers من نفس الـ Image من غير ما المساحة تتضرب في 10؟
2. اشرح اللي بيحصل لما تعدّل ملف موجود في الـ Image من جوه الـ Container.
3. مسحت ملف حجمه 500MB جوه Container، هل حجم الـ Image قلّ؟ وليه؟

الفصل السابع

07

الـ Volumes: البيانات اللي لازم تعيش

الـ Container Layer مؤقتة، بس فيه بيانات لازم تفضل موجودة حتى لو الـ Container اتمسح، زي قواعد البيانات والملفات اللي رفعها المستخدمين. الحل اسمه Volumes.

ليه محتاجين Volumes؟ | يعني إيه Volume في Docker؟ | أنواع الـ Volumes | أوامر إدارة الـ Volumes | لو كذا Container عدّلوا في نفس الوقت؟ | إزاي المهندسين بيحلّوا المشكلة دي؟

يعني إيه Database؟

هنتكلم في الفصل ده واللي بعده عن قواعد البيانات زي **Postgres** و MySQL كتير. الـ **Database** برنامج متخصص في حفظ بيانات التطبيق (المستخدمين والطلبات...) ورجّعها بسرعة، والتطبيق بيكلمها على Port (Postgres على 5432). وأهم حاجة هنا: بياناتها لازم تعيش حتى لو الـ Container اتمسح، وعشان كده هي أشهر مثال للـ Volumes. (اتشرحت في سطور في كتاب الإنترنت فصل 3.)

7.1 ليه محتاجين Volumes؟

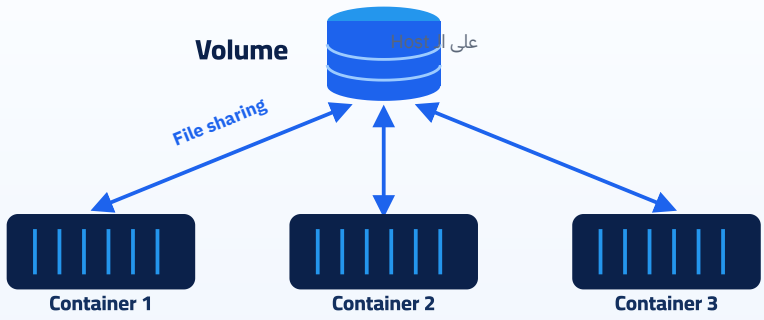
إحنا لحد دلوقتي فهمنا إن أي تعديل بنعمله جوه الـ Container بيتخزن في الـ Container Layer، والـ Layer دي مؤقتة. يعني أول ما الـ Container يتمسح أو يتشال، كل التعديلات اللي كانت فيها بتختفي. لكن في الاستخدام العملي، غالباً بيكون عندنا بيانات لازم تفضل موجودة حتى لو الـ Container اتشال أو اتعمله إعادة تشغيل.

بيانات قاعدة بيانات | ملفات رفعها المستخدمين | إعدادات | وغيرها

لو اعتمدنا على الـ Container Layer بس، كل البيانات دي هتضيع. علشان كده Docker قدّم مفهوم اسمه **Volumes**.

7.2 يعني إيه Volume في Docker؟

الـ Volume ببساطة هو طريقة تخلّي البيانات تتخزن **برّه** نظام الملفات الخاص بالـ Container. بدل ما البيانات تتحفظ جوه الـ Container Layer، بنخليها تتحفظ على جهازك انت نفسه (الـ Host). فبالتالي حتى لو الـ Container اتمسح، البيانات تفضل موجودة، ولما تشغّل Container جديد تقدر توصّله بنفس البيانات. فكأنك بتشارك ملفات معينة بينك وبين كل الـ Containers.

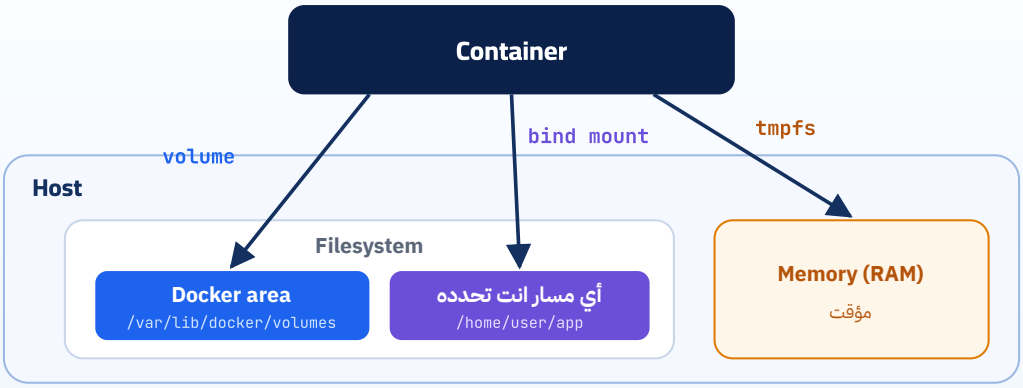


شكل 7.1 Volume واحد على الـ Host متشارك بين كذا Container.

> **معلومة مهمة**
>
> لازم تبقى فاهم إن التعديلات اللي هتتعمل من كل الـ Containers على الـ Volume هتفضل ثابتة. يعني الـ Volume نفسه، وهو على الـ Host، محتواه هيتحدّث ويتغير بناءً على التعديلات اللي بتحصل من كل الـ Containers، حتى بعد ما كل الـ Containers دي تقع.

ده يخلينا نسأل سؤال: لو حصل وكل الـ Containers عدّلت في نفس اللحظة على الـ Volume، إيه تأثير ده على محتوى الـ Volume؟ الإجابة: **"Docker ملوش دعوة، والبقاء للأخير"**. هنفهم الكلام ده في آخر الفصل.

7.3 أنواع الـ Volumes



شكل 7.2 كل نوع من أنواع التخزين بيحفظ البيانات فين على الـ Host.

Named Volumes (1)

ده أفضل وأأمن نوع، وأكتر نوع Docker بينصح بيه (**Best Practice**) في بيئة الإنتاج (Production).

```
Terminal
docker run -v my-volume:/path/in/container image_name
```

فكرته: انت بتطلب من Docker يعملك مساحة تخزين وبتديها "اسم محدد" (مثلاً `my-volume`). Docker بياخد الاسم ده، ويروح في مكان آمن وعميق جداً جوه ملفات النظام بتاعك (الـ Host)، ويعمل فولدر بالاسم ده ويديره بالكامل. ده المكان:

Host path

```
/var/lib/docker/volumes/my-volume/_data/
```

- عشان توصله لازم تبقى root، فاستخدم الأمر `sudo -i`. تقدر تضيف جواه ملفات أو فولدرات من الـ host عادي (وانت root)، أو من جوه الـ Container وهو شغال.
- لاحظ إن المكان ده `_data/` هو اللي بيشوفه الـ Container بس، سواء هيقرا اللي فيه أو يضيف ملفات جديدة.

> **على الماك**
>
> المسار ده جوه الـ Linux VM بتاعة Docker Desktop، مش على الماك نفسه، فـ `sudo -i` مش هيوصّلك ليه. اتفرج على محتوى الـ Volume من جوه Container: `docker run --rm -it -v my-volume:/data busybox ls /data`. أما الـ Bind Mounts (تحت) فبتشاور على فولدرات حقيقية على الماك عادي.

إمتى نستخدمه؟ لحفظ بيانات قواعد البيانات (Databases)، أو لو عايز تشارك نفس الداتا بين أكتر من Container في نفس الوقت.

Anonymous Volumes (2)

ده الأخ الصغير للـ Named Volume، بيشتغل بنفس الطريقة بالظبط، بس من غير اسم!

فكرته: Docker بيعمل مساحة تخزين آمنة برضه، بس بدل ما انت تدّيله اسم، Docker بيدّيله اسم عبارة عن "كود مشفّر طويل جداً" (Hash).

بيتعمل إزاي؟ بيتعمل تلقائياً لو استخدمت الـ Instruction بتاعة `VOLUME` جوه الـ Dockerfile، أو عن طريق الأمر ده:

Terminal

```
docker run -v /path/in/container my-image
```

إمتى نستخدمه؟ نادراً ما بنستخدمه في بيئة العمل الحقيقية. بيُستخدم أكتر لعمل Back Up سريع أو لحفظ داتا مؤقتة انت مش مهتم ترجعلها باسمها بعدين.

Bind Mounts (3)

انت اللي بتمسك الدركسيون وبتتحكم في كل حاجة، و Docker ملوش دعوة بإدارة الملفات دي!

فكرته: انت بتجيب مسار صريح ومحدد على جهازك انت (مثلاً `/home/user/app/`)، وتربطه بمسار جوه الـ Container. أي تعديل في المسار ده على جهازك بيسمّع فوراً جوه الـ Container، والعكس صحيح.

Terminal

```
docker run -v /path/on/my/laptop:/app my-node-app
```

إمتى نستخدمه؟ ده بطل بيئة التطوير (Development). لما تكون بتكتب كود وعايز الـ Container يحس بالتعديلات اللي بتعملها على الـ Editor بتاعك (زي VS Code) في نفس اللحظة، من غير ما تعمل Build للـ Image من أول وجديد (Live Reloading).

tmpfs Mounts (4)

ده النوع السري والسريع جداً، اللي مابيكتبش أي حاجة على الهارد ديسك أصلاً!

فكرته: المساحة دي بتتخلق جوه الـ RAM (الذاكرة العشوائية) بتاعة جهازك الأساسي. بمجرد ما الـ Container يقف، الداتا دي بتتبخر وتختفي للأبد.

Terminal

```
docker run --tmpfs /app/secure-temp-data my-secure-app
```

إمتى نستخدمه؟

- **للأمان العالي:** لو عندك ملفات حساسة جداً (Passwords و API Keys و Secrets) ومش عايزها تتكتب على الهارد ديسك عشان محدش يسرقها.
- **للأداء العالي (Performance):** لو الكونتينر بيعمل عمليات حسابية معقدة وبيكتب ويقرا ملفات مؤقتة كتير جداً، فالـ RAM أسرع بكتير من الهارد.

النوع	بيتخزن فين؟	مين بيديره؟	بيعيش لحد إمتى؟	أحسن استخدام
Named Volume	باسم Docker area انت مختاره	Docker	لحد ما تمسحه بنفسك	Databases و Production
Anonymous Volume	باسم Docker area Hash	Docker	لحد ما يتمسح (أو مع `docker rm -v`)	داتا مؤقتة مش مهم اسمها
Bind Mount	أي مسار على جهازك	انت	زي أي ملف عندك	Development و Live Reloading
tmpfs	الـ RAM	Docker	لحد ما الـ Container يقف	Secrets و ملفات مؤقتة سريعة

جدول 7.1 مقارنة بين أنواع التخزين الأربعة

معلومة إضافية: صيغة --mount

فيه صيغة أوضح وأطول بتعمل نفس الحاجة، ومفضّلة في السكربتات لأنها صريحة:

Terminal

```
docker run --mount type=volume,src=my-volume,dst=/data image_name
docker run --mount type=bind,src=/home/user/app,dst=/app image_name
```

7.4 أوامر إدارة الـ Volumes

بما إن Docker هو اللي بيدير الـ Named والـ Anonymous Volumes، فوفّرلنا مجموعة أوامر عشان نتحكم فيهم (الأوامر دي مابتشتغلش مع الـ Bind Mounts لأنها ملفاتك انت الشخصية):

`docker volume create my-volume-name`	إنشاء Volume جديد (قبل ما تشغّل الـ Container).
`docker volume ls`	عرض كل الـ Volumes الموجودة على جهازك.
`docker volume inspect my-volume-name`	معرفة تفاصيل الـ Volume (مكانه فين بالظبط على جهازك؟). هيطلعلك ملف JSON فيه مسار الـ Mountpoint، وده المكان السري اللي Docker شايل فيه ملفاتك.
`docker volume rm my-volume-name`	مسح Volume معين. Docker مش هيرضى يمسح الـ Volume لو فيه Container بيستخدمه.
`docker volume prune`	بيمسح أي Volume مش مربوط بـ Container حالياً. مفيد جداً عشان تفضّي مساحة من الهارد.

> ⚠ **خلي بالك**
>
> قبل ما تدوس Enter على `docker volume prune`، اتأكد إنك مش محتاج الداتا القديمة دي، لأنها بتتمسح نهائياً.

7.5 لو كذا Container عدّلوا في نفس الوقت؟

لازم تفهم إن Docker مش بيعمل أي إدارة للصراعات (**Conflict Resolution**) جوه الـ Volume. الـ Volume في النهاية هو مجرد فولدر عادي جداً موجود على الهارد ديسك بتاع جهازك. Docker بيفتح الباب للـ Containers ويقولهم: "الفولدر أهو، اتعاملوا انتوا مع بعض". التعامل هنا بيبقى كأنه نظام تشغيل عادي جداً، فاتحين فيه برنامجين بيحاولوا يعدّلوا على نفس الفولدر. وهنا بيظهر عندنا سيناريوهين:

السيناريو الأول: بيعدّلوا في ملفات مختلفة

لو الـ Container الأول بيكتب بيانات في ملف اسمه `file_A.txt`، والـ Container التاني بيكتب في ملف اسمه `file_B.txt`، والاتنين جوه نفس الـ Volume.

النتيجة: مفيش أي مشكلة خالص. الـ Volume هيحتوي على الملفين، والتعديلات كلها هتتحفظ.

السيناريو التاني: بيعدّلوا في نفس الملف في نفس اللحظة

لو 2 Containers بيحاولوا يفتحوا ملف اسمه `data.json` ويعدّلوا فيه في نفس اللحظة بالظبط، بيحصل حاجة في البرمجة اسمها **Race Condition** (حالة السباق).

إيه اللي هيحصل؟ القاعدة هنا هي **"Last Write Wins"** (الكتابة الأخيرة هي اللي بتكسب). يعني الـ Container اللي هيعمل Save متأخر جزء من الثانية هو اللي تعديله هيفضل، وهيمسح تعديل الـ Container الأولاني تماماً!

خلي بالك: Data Corruption

ولو البرنامجين بيكتبوا في نفس الجزء من الملف في نفس الملّي ثانية، ممكن يحصل **Data Corruption** (تلف في البيانات)، والملف كله يبوظ ويبقى غير قابل للقراءة.

7.6 إزاي المهندسين بيحلّوا المشكلة دي؟

بما إن Docker مش هيحل المشكلة، الحل لازم ييجي مننا إحنا بطريقتين:

الحل الأول: الاعتماد على ذكاء التطبيق نفسه (Application-Level Locks)

لو انت مشغّل Data Base (زي Postgres أو MySQL) في كذا Container وبيقروا من نفس الـ Volume، الـ Data Base دي متبرمجة بذكاء إنها تعمل حاجة اسمها **File Lock** (قفل الملف). يعني لما Container (أ) يدخل يعدّل في الملف، بيحط عليه قفل. لو Container (ب) حاول يعدّل في نفس اللحظة، هيلاقي الملف مقفول، فهيستنى في طابور لحد ما (أ) يخلص ويشيل القفل.

(عشان كده مشاركة نفس فوليوم الـ Data Base بين كذا Container شغالين مش محبّذة أصلاً، وليها طرق تانية أعقد زي الـ Replication).

الحل التاني: تحديد الصلاحيات (Read-Only)

دي الطريقة الأشهر والأأمن في Docker. لو عندك فولدر فيه صور أو ملفات، وعندك Container واحد وظيفته "يرفع" الصور دي، و 3 Containers تانيين وظيفتهم "يعرضوا" الصور دي للناس.

بنعمل إيه؟ بندّي للـ Container الأولاني صلاحية الكتابة، وبندّي للباقيين صلاحية "القراءة فقط"، وده بيتعمل بإضافة `:ro` في آخر الـ Volume:

```
Terminal
docker run -d --name uploader -v photos:/data       uploader-app
docker run -d --name viewer1  -v photos:/data:ro    viewer-app
docker run -d --name viewer2  -v photos:/data:ro    viewer-app
```

ملخص الفصل

- الـ Volume بيخزّن البيانات برّه الـ Container، فبتفضل موجودة حتى لو الـ Container اتمسح.
- Named Volume للإنتاج وقواعد البيانات، و Anonymous للمؤقت، و Bind Mount للتطوير، و tmpfs للـ RAM والـ Secrets.
- أوامر الإدارة: `create` و `ls` و `inspect` و `rm` و `prune` مع `docker volume`.
- Docker مش بيحل التعارض: الكتابة الأخيرة هي اللي بتكسب (Last Write Wins).
- الحل: Locks في التطبيق نفسه، أو صلاحيات قراءة فقط بـ `:ro`.

؟ أسئلة مراجعة

1. إيه الفرق بين `-v data:/app` و `-v /home/me/data:/app` ؟
2. انت بتطوّر موقع وعايز التعديلات تبان على طول جوه الـ Container، تستخدم أنهي نوع؟
3. فين بيتخزن الـ Named Volume على الـ Host؟
4. اشرح معنى "Last Write Wins" بمثال.
5. إزاي تخلّي Container يقرا من Volume من غير ما يقدر يعدّل فيه؟

الفصل التامن

Docker Networks: إزاي الـ Containers بتكلّم بعض

عرفنا إن كل Container ليه شبكة خاصة بيه، وفتحنا بورت عشان نوصله من جهازنا. طب لو عندي Backend عايز يكلم Database في Container تاني؟ هنا بييجي دور الـ Docker Networks.

يعني إيه Docker Network؟ | أنواع الشبكات: bridge و host و none | الشبكات اللي بتعملها بنفسك (User-defined) | جوه الشبكة: كلّم الـ Container باسمه بالـ DNS

إضافة

الفصل ده **إضافة** على الشرح الأصلي علشان الكتاب يبقى كامل، ومكتوب بنفس الأسلوب.

8.1 يعني إيه Docker Network؟

فاكر لما قلنا في مثال nginx إن الـ Container يعتبر جهاز خاص فأكيد ليه شبكة خاصة بيه؟ Docker بيعمل شبكات افتراضية جوه الـ Host، وكل Container بيتوصّل بشبكة (أو أكتر)، وبياخد IP خاص بيه جواها. الـ Containers اللي على نفس الشبكة يقدروا يكلّموا بعض، واللي على شبكات مختلفة معزولين عن بعض.

كل شبكة ليها **Driver** بيحدد هي بتشتغل إزاي. أهمهم تلاتة بيبقوا موجودين من أول ما تسطّب Docker:

```
Terminal

moaz@Linux:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
7b3c2a1f9d0e   bridge    bridge    local
1e5f8a2b6c3d   host      host      local
9a0d4c7e2f1b   none      null      local
```

8.2 أنواع الشبكات: bridge و host و none

الـ Driver	بيعمل إيه؟	إمتى نستخدمه؟
bridge (الافتراضي)	شبكة داخلية معزولة على الـ Host. كل Container بياخد IP جواها، وعشان توصله من بره لازم `-p`.	أغلب الحالات على سيرفر واحد.
host	الـ Container بيستخدم شبكة الـ Host نفسها مباشرة، من غير عزل ولا Port Mapping. لو nginx بيسمع على 80، يبقى على 80 بتاعة الـ Host على طول.	لما تحتاج أعلى أداء للشبكة (متاح على Linux).
none	مفيش شبكة خالص، غير الـ loopback جوه الـ Container.	مهام معزولة تماماً مش محتاجة نت.

جدول 8.1 الـ Network Drivers الأساسية

```
Terminal
docker run -d nginx                    # bridge (default)
docker run -d --network host nginx     # host network
docker run --rm --network none busybox ip addr
```

> **معلومة مهمة**
>
> فيه Drivers تانية للحالات المتقدمة، زي **overlay** اللي بيربط Containers على كذا سيرفر (بيُستخدم مع Docker Swarm)، و **macvlan** اللي بيدّي الـ Container عنوان MAC كأنه جهاز حقيقي على الشبكة.

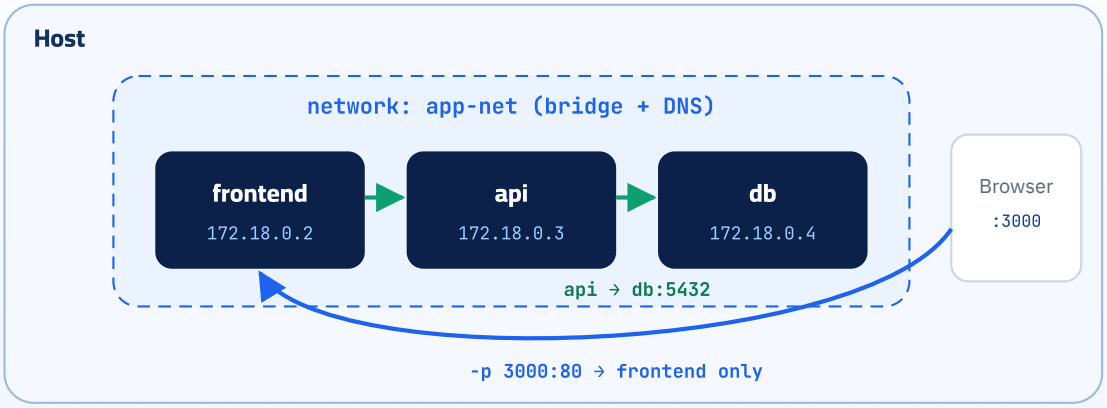
8.3 الشبكات اللي بتعملها بنفسك (User-defined)

أي Container بتشغّله من غير ما تحدد شبكة بيروح على الـ **bridge** الافتراضية. المشكلة إن الشبكة الافتراضية دي مفيهاش **DNS**، يعني الـ Containers يقدروا يكلّموا بعض بالـ IP بس، والـ IP بيتغيّر كل ما الـ Container يتعمل من جديد. الحل إنك تعمل شبكة بنفسك:

`docker network create app-net`	إنشاء شبكة جديدة من نوع bridge اسمها `app-net`.
`docker run -d --name db --network app-net postgres:16`	تشغيل Container وتوصيله بالشبكة من البداية.
`docker network connect app-net web`	توصيل Container شغال أصلاً بشبكة (و `disconnect` لفصله).
`docker network inspect app-net`	تفاصيل الشبكة: الـ Subnet والـ Containers المتوصّلة بيها.
`docker network rm app-net`	مسح الشبكة (لازم مفيش Containers عليها). و `docker network prune` يمسح كل الشبكات اللي مش مستخدمة.

8.4 الـ DNS جوه الشبكة: كلّم الـ Container باسمه

أحلى حاجة في الـ User-defined networks إن Docker بيشغّل فيها **DNS داخلي**، فأي Container يقدر يوصل لأي Container تاني على نفس الشبكة **باسمه** (الـ `--name`). يعني الـ Backend يكتب في إعداداته إن الـ Database موجودة على `db:5432`، ومش محتاج يعرف أي IP.



شكل 8.1 شبكة User-defined: الـ Containers بتكلّم بعض بالاسم، وبس اللي محتاج يتشاف من برّه هو اللي ليه Port Mapping.

مثال عملي: جرّب الـ DNS بنفسك

```
Terminal
moaz@Linux:~$ docker network create app-net
moaz@Linux:~$ docker run -d --name web --network app-net nginx
moaz@Linux:~$ docker run --rm --network app-net busybox wget -qO- http://web
<!DOCTYPE html>
<html><head><title>Welcome to nginx!</title> ...
```

الـ busybox وصل لـ nginx بكلمة `web` بس، من غير IP ومن غير `-p`، لأنهم على نفس الشبكة. (و `--rm` معناها امسح الـ Container أول ما يخلص).

خلي بالك

- الـ Containers اللي على نفس الشبكة بيكلّموا بعض على **بورت الـ Container** نفسه (زي 80 أو 5432)، مش على البورت اللي عملتله Mapping على الـ Host.
- متعملش `-p` لقاعدة البيانات لو مفيش حد من برّه محتاجها. سيبها جوه الشبكة بس، ده أأمن.

ملخص الفصل

- كل Container بيتوصّل بشبكة وبياخد IP جواها، والـ Driver الافتراضي هو bridge.
- host: الـ Container بيستخدم شبكة الـ Host مباشرة، و none: مفيش شبكة.
- اعمل شبكة بنفسك بـ `docker network create` علشان تاخد DNS داخلي.
- جوه الـ User-defined network الـ Containers بتكلّم بعض بالاسم وعلى بورت الـ Container.
- `-p` بس للحاجة اللي لازم تتشاف من برّه الـ Host.

؟ أسئلة مراجعة

1. ليه مينفعش تعتمد على الـ IP بتاع الـ Container في إعدادات التطبيق؟
2. إيه الفرق بين الـ bridge الافتراضية والـ bridge اللي انت بتعملها بنفسك؟
3. Container بـ `--network host` هل محتاج `-p` ؟ وليه؟
4. الـ api عايز يوصل لـ postgres اسمه `db` على نفس الشبكة، يكتب العنوان إزاي؟

09

الفصل التاسع

الـ Dockerfile: وصفة بناء الـ Image

هنا بنتعلم نكتب الـ Dockerfile بنفسنا: إزاي ننشئه، شكله العام، وكل Instruction بالتفصيل، مع الفروق المهمة والـ Best Practices اللي بتفرق بين Image محترمة و Image مليانة مشاكل.

يعني إيه Dockerfile؟ | بنستخدم Dockerfile ليه؟ | أهمية الـ Dockerfile: توثيق مش بس بناء | إزاي تنشئ Dockerfile | الشكل العام لـ Dockerfile | FROM: نقطة البداية | LABEL: بيانات وصفية

9.1 يعني إيه Dockerfile؟

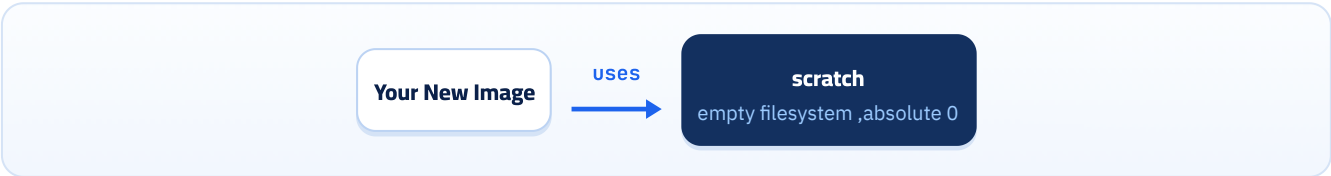
- زي ما احنا عارفين، الـ Dockerfile هو ملف نصّي بنكتب فيه مجموعة **Instructions** اللي Docker بيستخدمها عشان يبني Image.
- بمعنى أبسط، كل سطر انت بتكتبه جوه الـ Dockerfile بيكون خطوة من خطوات بناء الـ Image اللي هتطلع منه لما نعمله build.
- وعشان كده نقدر نقول إن الـ Dockerfile هو الأساس اللي بتتبني عليه أي Docker Image، لأنه هو اللي بيحدد بالظبط الـ Image دي هيبقى فيها إيه وإزاي هتتجهز.

9.2 بنستخدم Dockerfile ليه؟

الـ Dockerfile بيُستخدم غالباً في حالتين أساسيين:

(1) إنك تبني Image جديدة من البداية (From Scratch)

يعني تعمل Image من الصفر خالص، من غير ما تعتمد على Image موجودة قبل كده. (ودي تعتبر مستوى متقدم جداً جداً، نادراً ما بيحصل، أو نادراً ما بيتعامل معاه مبتدئ زيك).



شكل 9.1 بناء Image من الصفر مباشرة على `scratch`.

(2) إنك تعدّل على Image موجودة بالفعل

بمعنى إنك تاخد Image جاهزة وتستخدمها كقاعدة (**Base Image**)، وبعد كده تضيف عليها حاجات أو تغيّر فيها، بحيث تطلع Image جديدة مخصصة (Customized) تناسب احتياجاتك أو احتياجات الشغل. (ودي اللي هنركز عليها في الشرح أكتر لأنها الأكثر شيوعاً).



شكل 9.2 أي Image جاهزة هي في الآخر مبنية فوق Images تانية لحد ما توصل لـ `scratch`.

9.3 أهمية الـ Dockerfile: توثيق مش بس بناء

غير إنه بيبني الـ Image، الـ Dockerfile كمان ليه دور مهم جداً في **توثيق** طريقة بناء الـ Image، لأن كل الخطوات اللي حصلت أثناء البناء بتكون مكتوبة فيه بشكل واضح. وده معناه إن أي حد يشوف الـ Dockerfile يقدر يفهم:

- الـ Image دي اتبنت إزاي.
- إيه التعديلات اللي اتعملت عليها.
- وإيه المكونات أو الإعدادات الموجودة جواها.

> ⚠ **خلي بالك: docker commit**
>
> ده بيختلف عن إنك تعدّل يدوياً جوه Container وبعدين تعمل `docker commit`، لأن الطريقة دي بتطلّع Image جديدة فعلاً، لكن مش بتوضّح الخطوات اللي اتعملت عشان نوصل للتغييرات دي. محدش هيعرف يكرّرها أو يراجعها.

خلينا نمشي الموضوع خطوة خطوة: الأول إزاي تنشئ Dockerfile، بعد كده الشكل العام للـ syntax، وبعدها نمسك كل instruction بالتفصيل والقيم المحتملة والملاحظات المهمة.

9.4 إزاي تنشئ Dockerfile

مجرد ملف نصّي عادي بدون أي extension، يعني اسمه يكون بالظبط: `Dockerfile`، مش `Dockerfile.txt` ولا `dockerfile`. هتنشئ الملف عادي جداً زي أي ملف:

Terminal

```
nano Dockerfile
```

9.5 الشكل العام للـ Dockerfile

الـ Dockerfile عبارة عن instructions مكتوبة سطر سطر. الشكل العام بيكون كده:

Dockerfile

الـ Base Image اللي هنبني عليها	`FROM base_image:tag`
بيانات وصفية (Metadata)	`LABEL key=value`
واحد منهم أو الاتنين مع بعض	`ENV variable=value` `ARG variable=value`

`WORKDIR /path`	مسار الشغل
`COPY source destination` `ADD source destination`	واحد منهم (COPY أو ADD)
`RUN command`	أوامر وقت البناء
`EXPOSE port`	توثيق البورت
`CMD ["command"]` `ENTRYPOINT ["command"]`	أمر التشغيل: CMD أو ENTRYPOINT

Best Practice: قلّل عدد الـ Layers

لاحظ إن كل سطر من سطور الـ Dockerfile بيتحوّل لـ **layer** لما يتعمل build للـ Dockerfile ويبقى image. فلذلك مهم جداً نقلّل عدد السطور، عشان منخليش عدد الـ layers بتاعة الـ image اللي هتطلع كبير، لأنه هينعكس على حجم الـ image ويزوّدها، وده مش مستحب خالص.

فلو مثلاً هضطر أكتب نفس الـ instruction أكتر من مرة، هحاول أجمعهم كلهم في نفس السطر مع بعض عشان يبقوا layer واحدة، بدل ما أكتبهم في كذا سطر ويطلع عندي كذا layer والحجم يزيد.

معلومة مهمة

بالتحديد، الـ Instructions اللي بتضيف ملفات فعلاً (`RUN` و `COPY` و `ADD`) هي اللي بتعمل Layers ليها حجم. الباقي (زي `ENV` و `LABEL` و `CMD`) بيضيف معلومات (metadata) بس، ومبيكبّرش الحجم.

9.6 FROM: نقطة البداية

ده أول سطر تقريباً في أي Dockerfile. وظيفته يحدد الـ **base image** اللي هنبني عليها.

Syntax

```
FROM image_name:tag
```

`image_name`	اسم الـ image اللي عايز تبني عليها (لو هتبني image جديدة من image قديمة)، أو `scratch` لو عايز تعمل image جديدة من الصفر، وكده معناه إن مفيش OS جاهز.
`tag`	الـ version بتاع الـ image. لو مكتبتش `:tag` خالص هيفترض إنك عايز أحدث نسخة (يعني كأنك كتبت `:latest`).

9.7 LABEL: بيانات وصفية

طريقة بنضيف بيها "بيانات وصفية" (**Metadata**) للـ Docker Image بتاعتنا. البيانات دي مبتأثرش خالص على طريقة شغل الـ Image أو الكود اللي جواها، لكنها بتدّي معلومات توضيحية مهمة جداً للمطورين أو للأنظمة اللي بتتعامل مع الـ Image دي.

```
Syntax
LABEL key=value
LABEL key1=value1 key2=value2 ... keyN=valueN
LABEL author="moaz"
```

ولو عايز تكتب كذا حاجة جوه نفس السطر بتاع الـ instruction بس عايز تقسّمهم لسطور تحت بعض، دي الطريقة العامة (علامة `\` في آخر السطر):

```
Dockerfile
LABEL key1=value1 \
      key2=value2 \
      keyN=valueN
```

9.8 ENV و ARG: المتغيرات

ENV

بتُستخدم لتعريف متغيرات البيئة (**Environment Variables**). الفكرة هنا إنك بتحدد قيم ثابتة أو إعدادات معينة (زي الـ Port، أو مسار ملفات، أو بيئة التشغيل) جوه الـ Image. أهم ميزة في الـ ENV إن المتغيرات دي بتفضل موجودة وشغالة حتى بعد ما الـ Image تتحول لـ Container ويبدأ يشتغل، وتقدر تغيّر قيمتها من برّه وانت بتشغّل الـ Container (بـ `-e`)، من غير ما تحتاج تبني الـ Image من أول وجديد. (والعُرف إن أسماء المتغيرات تتكتب كلها Capital).

```
Syntax
ENV key=value
ENV key1=value1 key2=value2 ... keyN=valueN
ENV PORT=8080
```

ARG

هو متغير بنستخدمه عشان نمرّر قيم أو إعدادات للـ Dockerfile **وقت بناء الـ Image فقط**. الفرق الجوهري بينه وبين الـ ENV إن الـ ARG بيختفي تماماً بمجرد ما الـ Image تخلص بناء، ومبيكونش متاح جوه الـ Container وهو شغال.

```
Syntax
ARG key
ARG key=value
ARG key1=value1 key2=value2 ... keyN=valueN
```

	ENV	ARG
متاح وقت الـ build	أيوه	أيوه
متاح جوه الـ Container	أيوه، وبيفضل موجود	لأ، بيختفي بعد الـ build
تغيير القيمة	`docker run -e PORT=9090`	`docker build --build-arg VERSION=2`
مثال استخدام	PORT و NODE_ENV	رقم version لأداة هتتسطّب

جدول 9.1 ENV مقابل ARG

> **خلي بالك**
> متحطش Passwords أو API Keys في `ENV` أو `ARG` جوه الـ Dockerfile، لأنهم بيتسجّلوا في الـ Image وأي حد معاه الـ Image يقدر يشوفهم بـ `docker inspect` أو `docker image history`.

9.9 WORKDIR: مسار الشغل

بيحدد "مسار العمل الحالي" (**Current Working Directory**). المعنى هنا إن أي Instruction هتيجي تحته في الـ Dockerfile هتتنفذ تلقائياً جوه المسار اللي انت هتحدده.

Syntax

```
WORKDIR /path
```

كأنك بتخلّي قبل كل Command ممكن يتكتب جوه الـ Container وهو شغال، يتعمل الأمر `cd /path` على المسار اللي حددته. (ولو المسار مش موجود، Docker بيعمله لوحده).

9.10 COPY و ADD: نقل الملفات

بياخد نسخة من الملفات أو الفولدرات اللي موجودة على جهازك (Host)، ويحطها جوه الـ Docker Image، فالملفات أو الفولدرات دي هتبقى موجودة جوه الـ Container وهو شغال.

Syntax

```
COPY source_path destination_path
ADD  source_path destination_path
```

الـ ADD والـ COPY بيعملوا نفس الحاجة بالظبط، بس الـ ADD معاه حاجتين زيادة مابيعملهمش COPY:

(1) فك الضغط التلقائي (Auto-Extraction)

لو المصدر (src) بتاعك عبارة عن ملف مضغوط (زي `.tar` أو `.tar.gz`) موجود على جهازك، Docker مش بس هينقله، ده هيفك ضغطه تلقائياً جوه الـ Image في المسار اللي انت حددته.

```Dockerfile
ADD my_app_code.tar.gz /app/
```

(2) التحميل من الروابط (URL Download)

لو المصدر (src) عبارة عن لينك (URL) لملف على الإنترنت، Docker هيحمّله ويحطه في الوجهة اللي حددتها.

```Dockerfile
ADD https://example.com/config.json /app/
```

Best Practice: إمتى أستخدم ده وإمتى ده؟

توصيات Docker الرسمية بتقولك: **"استخدم COPY دايماً، إلا للضرورة القصوى!"**. طب ليه بنفضّل COPY؟

- **الوضوح والشفافية:** لما المطور بيشوف COPY بيبقى عارف ومتأكد إن دي مجرد عملية نقل ملفات بسيطة. لكن ADD ممكن يكون بيفك ضغط ملف أو بيحمّل حاجة من النت، وده بيخلّي سلوك الـ Dockerfile مش متوقع أحياناً.
- **ميزة التحميل من الروابط في ADD مش مفضّلة:** لأنك لو حمّلت ملف بـ ADD مش هتقدر تمسح الملفات المؤقتة في نفس الخطوة (يعني الملف الأصلي المضغوط هيفضل موجود حتى بعد ما يتفك ضغطه)، وده بيكبّر حجم الـ Image على الفاضي، لأنه هيبقى فيها الملف المضغوط + الملفات اللي طلعت منه.

البديل الأفضل لتحميل الروابط: بدل ما تستخدم ADD لتحميل ملف من النت، الأفضل تستخدم RUN مع أداة زي `curl` أو `wget`، عشان تقدر تحمّل الملف، وتعمل اللي انت عايزه، وتمسح الملفات الزيادة في نفس السطر (وبكده تحافظ على حجم الـ Image صغير):

```Dockerfile
RUN curl -O https://example.com/big_file.tar.gz \
    && tar -xzf big_file.tar.gz \
    && rm big_file.tar.gz
```

الميزة	COPY	ADD
نسخ ملفات وفولدرات من الجهاز	أيوه	أيوه
فك ضغط `.tar` تلقائي	لأ	أيوه
تحميل من URL	لأ	أيوه (مش مفضّل)
السلوك متوقع وواضح	دايماً	مش دايماً
التوصية	**الاختيار الافتراضي**	للضرورة بس (فك ضغط ملف محلي)

جدول 9.2 COPY مقابل ADD

9.11 RUN: عضلات الـ Dockerfile

هي "العضلات" بتاعة الـ Dockerfile. دي الـ Instruction اللي بتنفّذ أي أمر انت متعوّد تكتبه بإيدك في التيرمينال (زي تسطيب Packages، أو تنزيل Libraries، أو إنشاء فولدرات).

بيشتغل إمتى؟ بيشتغل وقت بناء الـ Image فقط (**Build-Time**). بمجرد ما الـ Image تتبني، الأوامر دي مش بترجع تشتغل تاني جوه الـ Container.

Syntax

```
RUN command
RUN command1 && command2 && ...
```

Best Practice: عشان تحافظ على حجم الـ Image صغير وسريع

- **دمج الأوامر (Chaining):** دايماً ادمج أوامر التحديث، والتسطيب، وتنظيف الملفات المؤقتة في سطر واحد (طبقة واحدة) باستخدام `&&` وعلامة `\` عشان تنزل سطر جديد.
- **فخ الكاش (The Cache Trap):** إياك تكتب أمر التحديث (`apt-get update`) في سطر RUN منفصل. Docker هيستخدم الكاش (النسخة المخبّأة) ومش هيحدّث حاجة. دايماً اربط التحديث بالتسطيب.

Dockerfile

```
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
```

9.12 CMD و ENTRYPOINT: أمر التشغيل

هما التعليمات اللي بتحدد "البرنامج أو الـ Process الأساسي" اللي الـ Container هينفّذه أول ما يشتغل (**Run-time**). حياة الـ Container مرتبطة بالبرنامج ده: طول ما هو شغال الـ Container عايش، ولما يخلص (أو يقفل) الـ Container بيقع (Exits) فوراً.

Syntax

```
CMD        ["executable", "param1", "param2"]
ENTRYPOINT ["executable", "param1", "param2"]
```

ملاحظة: الـ Exec Form

اللي بيتكتب بعدهم هو command عادي، ولكن مش بيتكتب بالصورة العادية (كلمات بينها مسافات)، لأ بيتكتب بـ **Exec Form**، يعني كل كلمة بتتحط بين علامتين تنصيص مزدوجة `"..."`، وكلهم جوه `[ ]`.

الـ CMD والـ ENTRYPOINT بيعملوا نفس الحاجة بالظبط (تشغيل الكونتينر)، بس الفرق الجوهري بينهم في **"المرونة مقابل الإجبار"**:

(1) الأمر الافتراضي المرن: CMD

بيحدد أمر افتراضي. لو المستخدم وهو بيشغّل الـ Container بـ `docker run` كتب أي أمر إضافي من عنده، Docker بيلغي أمر الـ CMD تماماً، وبينفّذ أمر المستخدم.

Dockerfile

```
CMD ["echo", "Hello"]
```

Terminal

```
moaz@Linux:~$ docker run my-image bash
# الـ Container هيتجاهل echo ويشغّل bash
```

(2) الأمر الإجباري الثابت: ENTRYPOINT

بيحدد برنامج ثابت وإجباري، مستحيل يتلغي بمجرد كتابة أمر بعد `docker run`. لو المستخدم كتب أي كلام زيادة، Docker بياخد الكلام ده ويعتبره "إضافات" (Arguments)، ويلزقه جنب أمر الـ ENTRYPOINT الأساسي.

Dockerfile

```
ENTRYPOINT ["echo", "Hello"]
```

Terminal

```
moaz@Linux:~$ docker run my-image World
Hello World
```

Best Practice: إمتى أستخدم ده وإمتى ده؟

- **استخدم CMD:** لو بتعمل Image عامة وممكن تشغّل جواها كذا حاجة (زي بيئة Ubuntu أو Node)، ومحتاج بس تدّي أمر افتراضي ينفع المستخدم يغيّره بسهولة.
- **استخدم ENTRYPOINT:** لو بتعمل Image معمولة لغرض واحد محدد (زي سيرفر Nginx أو أداة CLI زي `curl` أو `ping`)، وعايز تقفل الكونتينر على البرنامج ده عشان محدش يلغيه بالغلط.

دمجهم مع بعض (The Pro Way)

في بيئات العمل الحقيقية، أحسن طريقة إننا نستخدمهم هما الاتنين مع بعض: بنخلّي الـ ENTRYPOINT هو البرنامج الأساسي (اللي مش هيتغير)، والـ CMD هي الإضافات الافتراضية (اللي ينفع تتغير لو اليوزر حابب).

Dockerfile

```
ENTRYPOINT ["ping"]
CMD ["google.com"]
```

Terminal

```
moaz@Linux:~$ docker run my-ping
ping google.com #
moaz@Linux:~$ docker run my-ping docker.com
ping docker.com #
```

	CMD	ENTRYPOINT
دوره	أمر افتراضي	البرنامج الثابت للـ Container
لو كتبت أمر بعد `docker run`	بيتلغي ويتنفّذ أمرك	كلامك بيتضاف كـ Arguments
تغييره بالإجبار	بسهولة	بس بـ `--entrypoint`
مناسب لـ	Images عامة (ubuntu و node)	Images لغرض واحد (nginx و أدوات CLI)

جدول 9.3 CMD مقابل ENTRYPOINT

9.13 EXPOSE: يافطة معلومات

بتمثّل "يافطة معلومات" (**Documentation**). بنستخدمها عشان نقول للمطورين اللي هيستخدموا الـ Image من بعدنا: "يا جماعة، الـ Container ده لما يشتغل، هيكون جواه تطبيق مستني حد يكلّمه على الـ Port الفلاني".

Syntax

```
EXPOSE port
```

> ⚠ **الخدعة الكبيرة: إيه اللي EXPOSE مابيعملوش؟**
>
> الـ EXPOSE **مش بيفتح الـ Port فعلياً** للعالم الخارجي (لجهازك). يعني لو كتبت `EXPOSE 80` وشغّلت الـ Container، مش هتقدر تفتح موقعك من متصفح جهازك. هي مجرد توثيق داخلي بيقول إن الـ Container جاهز يستقبل اتصالات على الـ Port ده جوه شبكة Docker نفسها.

إزاي نفتح الـ Port بجد؟ زي ما عملنا في مثال nginx: وانت بتعمل run للـ image هتضيف بعد كلمة run:

Terminal

```
-p host_port:container_port
```

أو `-P` بس، وساعتها هيختار الـ Port بشكل عشوائي لكل بورت متعمله EXPOSE.

9.14 USER: متشغّلش كـ root

بيحدد مين هو "المستخدم" (User) اللي هينفّذ الأوامر اللي جاية في الـ Dockerfile (تحديداً RUN و CMD و ENTRYPOINT)، وهو نفس المستخدم اللي الـ Container هيشتغل بصلاحياته بعد كده.

Syntax

```
USER user_name
USER UID
```

لو مكتبتش USER خالص، Docker بيشغّل الـ Container والـ App بتاعك بصلاحيات الـ **root** (المدير العام اللي يقدر يعمل أي حاجة). وده خطر جداً أمنياً! طب ليه؟ تخيّل لو تطبيقك فيه ثغرة والـ Hacker قدر يخترق الـ Container. لو الـ Container شغال بـ root، الـ Hacker ممكن يسيطر على جهازك الأساسي (Host) نفسه! لكن لو شغال بمستخدم عادي، الـ Hacker هيكون محبوس بصلاحيات ضعيفة.

طب إيه الحل؟ نعمل خدعة بسيطة: بنسطّب البرامج الأول كـ root، وبعدين بنعمل مستخدم جديد بصلاحيات محدودة، وفي الآخر بنعمل سويتش (Switch) للمستخدم ده عشان نشغّل بيه التطبيق:

Dockerfile

1 إحنا هنا root (عشان نعرف نسطّب برامج). مش شرط تعمل كده بالظبط، بس القصد إنك تخلّص كل اللي محتاج تعمله وانت root، وبعدها تضيف في الآخر أمر بيعمل user جديد.	`RUN apk add --no-cache nodejs \ && adduser -D appuser`
2 بنعمل Switch للمستخدم الجديد.	`USER appuser`
3 الأمر ده هيشتغل بصلاحيات `appuser` الضعيفة (أمان تام).	`CMD ["node", "app.js"]`

ملخص الفصل

- الـ Dockerfile ملف اسمه `Dockerfile` بالظبط، فيه Instructions بتبني الـ Image، وبيوثّق طريقة بنائها.
- FROM للبداية، و LABEL للـ Metadata، و WORKDIR لمسار الشغل.
- ENV بيفضل جوه الـ Container، و ARG وقت الـ build بس.
- COPY هو الافتراضي، و ADD للضرورة (فك ضغط ملف محلي).
- RUN وقت الـ build: ادمج الأوامر ونضّف في نفس السطر.
- CMD أمر افتراضي مرن، و ENTRYPOINT برنامج ثابت، والأحسن الاتنين مع بعض.
- EXPOSE توثيق بس، والفتح الحقيقي بـ `-p`. و USER علشان متشغّلش كـ root.

أسئلة مراجعة

1. ليه الأفضل تكتب الـ Image بـ Dockerfile بدل `docker commit`؟
2. اكتب سطر RUN واحد يحدّث الـ packages ويسطّب `git` وينضّف الكاش.
3. Image فيها `ENTRYPOINT ["ping"]` و `CMD ["google.com"]`، إيه اللي هيتنفّذ لو كتبت `docker run img 8.8.8.8`؟
4. كتبت `EXPOSE 3000` ومش عارف تفتح التطبيق من المتصفح، ليه؟ وإيه الحل؟
5. إيه الفرق بين ENV و ARG؟ وإمتى تستخدم كل واحد؟
6. ليه تشغيل الـ Container كـ root خطر؟

10

الفصل العاشر

بناء الـ Image و Multi-stage Build

خلّصنا كتابة الـ Dockerfile، فاضل نعمله build ونشغّل الناتج. وبعدها نشوف مشكلة الـ Images التقيلة اللي مليانة كراكيب، وحلها الذكي: Multi-stage Build.

أمر docker build | مثال عملي: موقع static بـ nginx | المشكلة: الـ Image المليانة كراكيب | الحل: Multi-stage Build (نظرية المطبخ وصالة الطعام) | مثال عملي متكامل: تطبيق React مع Nginx | تحليل السحر اللي حصل هنا | إيه المكاسب اللي طلعنا بيها؟

10.1 أمر docker build

بمجرد ما تخلّص كتابة الـ Dockerfile، تحفظه وتقفله، وبعدها تكتب:

`docker build -t image_name /path/to/folder`	ابني Image من الفولدر ده، اللي جواه ملف اسمه `Dockerfile`.
`docker build -t image_name .`	لو الـ Dockerfile في نفس المسار اللي انت فيه (النقطة = المسار الحالي).
`docker build -t image_name -f /path/to/MyDockerfile .`	لو الملف اسمه أي حاجة غير `Dockerfile`، تحدد مكانه بـ `-f`.

يعني إيه النقطة في الآخر؟

المسار اللي في آخر أمر `docker build` اسمه الـ **Build Context**: ده الفولدر اللي Docker بيبعته كله للـ builder، وهو اللي `COPY` و `ADD` بيقدروا ياخدوا منه ملفات. وعلشان كده هنحتاج ملف `.dockerignore` اللي هنشرحه في الفصل الجاي.

10.2 مثال عملي: موقع static بـ nginx

عايزين نعمل Dockerfile نعمل بيه Image جديدة تقوّملنا موقع باستخدام nginx، بس الموقع هيكون خاص بينا، مش الصفحة الافتراضية بتاعة nginx.

1. نعمل فولدر فيه ملفات الموقع اسمه `my-static-website`، ونعمل جواه ملفين اتنين بس: ملف الكود بتاع الموقع `index.html`، وملف الـ `Dockerfile`.
2. هنفتح ملف الـ Dockerfile ونكتب فيه (هتلاحظ إننا طبّقنا كل اللي شرحناه قبل كده):

Dockerfile

1 بنحدد البيئة الأساسية (اخترنا نسخة alpine لأن حجمها صغير جداً وسريعة).	`FROM nginx:alpine`
2 بنحدد مسار العمل الافتراضي اللي Nginx متبرمج يقرا منه ملفات الموقع.	`WORKDIR /usr/share/nginx/html`

3 بنمسح صفحة الترحيب الافتراضية بتاعة Nginx عشان نحط صفحتنا إحنا.	`RUN rm -rf ./*`
4 بننسخ ملف `index.html` من جهازنا لمسار العمل جوه الـ Image.	`COPY ./index.html .`
5 بنوثّق إن الـ Container بيستقبل الاتصالات على بورت 80.	`EXPOSE 80`

> **معلومة مهمة**
>
> مكتبناش CMD ولا ENTRYPOINT، لأننا ورثناهم جاهزين من الـ Image الأصلية بتاعة Nginx.

3 دلوقتي هنفتح التيرمينال (Terminal) جوه الفولدر بتاعنا، ونكتب أمر البناء:

```
Terminal
docker build -t my-nginx-website .
```

هنا إحنا بنقول لـ Docker: ابني Image واديها اسم `my-nginx-website` باستخدام ملف الـ Dockerfile اللي في المسار الحالي.

4 بعد ما البناء يخلص، هنشغّل الـ Container ونربط البورتات عشان نقدر نشوف الموقع:

```
Terminal
docker run -d --name my-running-site -p 8080:80 my-nginx-website
```

وافتح `http://localhost:8080` هتلاقي صفحتك انت.

10.3 المشكلة: الـ Image المليانة كراكيب

تخيّل إنك بتعمل تطبيق بـ React أو Angular، أو حتى برنامج بـ Java أو Go. عشان تحوّل الكود بتاعك لتطبيق شغال، انت محتاج الآتي:

- **أدوات البناء (Build Tools):** زي Node.js و npm أو المترجمات (Compilers).
- **كود المصدر (Source Code):** الكود اللي انت كاتبه.
- **المكتبات (Dependencies):** فولدر زي `node_modules` اللي حجمه ممكن يوصل لجيجات!

لو كتبنا Dockerfile عادي، هنضطر نحط كل ده جوه الـ Image. النتيجة؟

حجم الـ Image ضخم جداً

ممكن يوصل لـ 1 أو 2 جيجا! في حين إن التطبيق النهائي (ملفات HTML و CSS و JS) حجمه 10 ميجا بس.

كارثة أمنية (Security Risk)

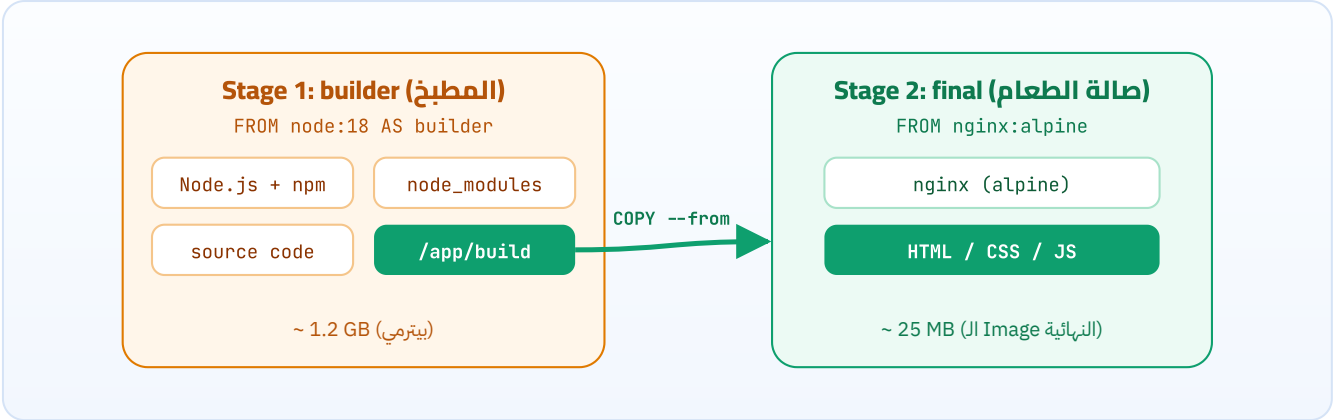
انت كده رافع الـ Source Code بتاعك، وأدوات البناء، والمكتبات على سيرفر الإنتاج (Production). لو هاكر دخل، هيلاقي بيئة كاملة يقدر يلعب فيها براحته!

10.4 الحل: Multi-stage Build (نظرية المطبخ وصالة الطعام)

فكرة البناء متعدد المراحل ببساطة عاملة زي المطعم:

- **المرحلة الأولى (المطبخ):** فيها الطبّاخين، والسكاكين، ومكونات الأكل الخام، والبوتاجاز. بنطبخ هنا الأكل (بنعمل Build للكود).
- **المرحلة التانية (صالة الطعام):** دي اللي الزبون بيقعد فيها. الزبون مش محتاج يشوف السكاكين ولا قشر البصل، هو محتاج "الطبق النهائي الجاهز" بس!

Docker بيسمحلك تعمل كذا `FROM` في نفس الـ Dockerfile. كل `FROM` بتبدأ مرحلة (**Stage**) جديدة تماماً. وفي النهاية، Docker بياخد المرحلة الأخيرة بس ويحوّلها لـ Image، ويرمي كل المراحل اللي قبلها في الزبالة!



شكل 10.1 الـ Multi-stage Build: بنبني في مرحلة، وبناخد الناتج بس للمرحلة الأخيرة.

10.5 مثال عملي متكامل: تطبيق React مع Nginx

Dockerfile

1 **المرحلة الأولى: مرحلة البناء (المطبخ).** بنستخدم نسخة Node عشان فيها أدوات البناء، وبندّي للمرحلة دي اسم `builder`.	`FROM node:18 AS builder` `WORKDIR /app`
2 بننسخ ملفات المكتبات وبنعملها Install.	`COPY package.json .` `RUN npm install`
3 بننسخ الكود بتاعنا كله وبنعمله Build (بيطلع فولدر اسمه `build` جواه التطبيق النهائي).	`COPY . .` `RUN npm run build`
4 **المرحلة التانية: مرحلة التشغيل (صالة الطعام).** بنبدأ مرحلة جديدة تماماً بنسخة Nginx خفيفة جداً (حجمها حوالي 20 ميجا).	`FROM nginx:alpine`
5 بنقول لـ Docker: روح للمرحلة اللي اسمها `builder`، هات منها الفولدر النهائي بس، وسيب الباقي!	`COPY --from=builder /app/build` `/usr/share/nginx/html`
6 بنوثّق البورت، ونشغّل nginx في الـ foreground علشان الـ Container يفضل عايش.	`EXPOSE 80` `CMD ["nginx", "-g", "daemon off;"]`

Best Practice: ترتيب السطور والكاش

لاحظ إننا نسخنا `package.json` وعملنا `npm install` **قبل** ما ننسخ باقي الكود. كده لو غيّرت في الكود بس، Docker هيستخدم الكاش للـ `npm install` ومش هيعيده، والـ build يبقى أسرع بكتير.

10.6 تحليل السحر اللي حصل هنا

تسمية المراحل (AS builder)

إحنا سمّينا الـ FROM الأولى `builder` عشان نقدر ننادي عليها بعدين. (تقدر تسمّيها أي اسم زي `build-stage`، أو حتى تعتمد على رقمها `0`).

بداية مرحلة جديدة (FROM nginx:alpine)

بمجرد ما كتبنا FROM تانية، Docker بدأ Image جديدة نظيفة تماماً. مفيش فيها Node.js، ولا كود المصدر، ولا `node_modules`.

الكوبري السحري (COPY --from)

إحنا محتاجين بس "الطبق الجاهز" اللي طلع من المطبخ، فاستخدمنا فلاج `--from=builder` مع COPY. ده بيخلّي Docker يروح للـ Image المؤقتة بتاعة المرحلة الأولى، وياخد فولدر الـ build (اللي فيه ملفات الـ HTML والـ JS الصافية)، وينسخه للـ Image الجديدة.

10.7 إيه المكاسب اللي طلعنا بيها؟

المكسب	التفاصيل
حجم Image خرافي (Small Footprint)	بدل ما الـ Image كان حجمها 1.2 جيجا، بقى حجمها 25 ميجا بس! (حجم الـ Nginx + ملفات الموقع). وده معناه سرعة رهيبة في الرفع (Push) والتنزيل (Pull) على السيرفرات.
أمان لا يُعلى عليه (High Security)	الـ Image النهائية مفيهاش غير سيرفر Nginx وملفات HTML و JS ثابتة. مفيش Node.js، ومفيش كود سري، ومفيش أدوات بناء ممكن حد يستغلها عشان يعمل اختراق. لو الهاكر دخل، مش هيلاقي حاجة يشتغل بيها أصلاً!
كود أنظف (Clean Dockerfile)	زمان كنا بنضطر نكتب سكربتات معقدة جداً جوه سطر RUN واحد عشان ننزّل أدوات البناء، ونعمل Build، وبعدين نمسح الأدوات دي تاني عشان نوفّر مساحة. دلوقتي كل حاجة بقت منظمة ومفصّلة.

جدول 10.1 المكاسب من الـ Multi-stage Build

ملخص الفصل

- `docker build -t name .` بيبني Image من الـ Dockerfile اللي في المسار الحالي، و `-f` لاسم ملف مختلف.
- المسار في آخر الأمر هو الـ Build Context.
- الـ Image العادية لتطبيق محتاج build بتبقى ضخمة وخطر أمنياً.
- Multi-stage: كذا `FROM`، والمرحلة الأخيرة بس هي اللي بتبقى Image، و `COPY --from` بينقل الناتج.
- رتّب السطور بحيث الحاجات اللي بتتغير قليل تيجي الأول علشان تستفيد من الكاش.

؟ أسئلة مراجعة

1. إيه معنى النقطة في `docker build -t app .` ؟
2. ليه مكتبناش CMD في مثال الموقع الـ static؟
3. في الـ Multi-stage، الـ Image النهائية فيها Node.js ولا لأ؟ وليه؟
4. ليه بننسخ `package.json` لوحده قبل باقي الكود؟

الفصل الحادي عشر

ملف .dockerignore: متبعتش الكراكيب للـ build

لما بتكتب COPY . . انت بتنسخ كل حاجة في الفولدر، بما فيها node_modules و .git وملفات الـ .env اللي فيها أسرار. ملف صغير اسمه .dockerignore بيحل المشكلة دي.

المشكلة: الـ Build Context بيتبعت كله | الحل: ملف .dockerignore | قواعد الكتابة | المكاسب

إضافة

الفصل ده **إضافة** على الشرح الأصلي علشان الكتاب يبقى كامل، ومكتوب بنفس الأسلوب.

11.1 المشكلة: الـ Build Context بيتبعت كله

فاكر النقطة اللي في آخر `docker build -t app .`؟ دي معناها إن الفولدر الحالي كله هو الـ **Build Context**، و Docker بيبعته للـ builder قبل ما يبدأ. ولما تكتب `COPY . .` في الـ Dockerfile، كل اللي في الفولدر بيتنسخ جوه الـ Image، ومن ضمنه حاجات انت مش عايزها خالص:

حاجات بتكبّر الحجم وتبطّأ الـ build

- `node_modules` (والـ Image هتعمل install لوحدها أصلاً)
- فولدر `.git` بكل تاريخ المشروع
- ملفات الـ logs والـ build القديمة

حاجات خطر تتسرّب

- ملفات `.env` فيها Passwords و API Keys
- مفاتيح SSH أو شهادات
- ملفات إعدادات الجهاز الشخصية

11.2 الحل: ملف .dockerignore

ملف نصّي اسمه بالظبط `.dockerignore` (بالنقطة في الأول)، بتحطه في **جذر الـ Build Context** (جنب الـ Dockerfile غالباً)، وبتكتب فيه الملفات والفولدرات اللي Docker يتجاهلها. فكرته نفس فكرة `.gitignore` بالظبط.

.dockerignore

```
# dependencies (هتتسطّب جوه الـ Image)
node_modules
npm-debug.log

# git و إعدادات المحرر
.git
.gitignore
.vscode

# أسرار
.env
*.pem

# ناتج build قديم و logs
build
dist
*.log

# متتجاهلش الملف ده
!.env.example
```

11.3 قواعد الكتابة

الصيغة	معناها	مثال
`name`	تجاهل الملف أو الفولدر ده	`node_modules`
`*`	أي مجموعة حروف في نفس المستوى	`*.log`
`**`	أي عدد من الفولدرات	`**/*.tmp`
`!`	استثناء: متتجاهلش ده	`!.env.example`
`#`	تعليق	`# secrets`

جدول 11.1 الـ Patterns في ملف .dockerignore.

11.4 المكاسب

- **Build أسرع:** الـ Context بيبقى صغير، فبيتبعت أسرع.
- **Image أصغر:** مفيش ملفات ملهاش لازمة جوه الـ Image.
- **كاش أحسن:** أي تعديل في `.git` أو ملف log مش هيبوّظ الكاش بتاع `COPY . .`.
- **أمان:** ملفات الأسرار متدخلش الـ Image أصلاً، فمتتسرّبش لما ترفعها على Docker Hub.

خلي بالك

الـ `.dockerignore` بيتقري من جذر الـ Build Context بس. لو حطيته في فولدر فرعي أو سمّيته غلط (زي `dockerignore` من غير نقطة) مش هيشتغل، ومش هيطلعلك أي Error.

Best Practice

اعمل `.dockerignore` مع أول Dockerfile في أي مشروع، خصوصاً لو بتستخدم `COPY . .` زي مثال الـ React في الفصل اللي فات.

ملخص الفصل

- الـ Build Context هو الفولدر اللي في آخر أمر `docker build`، وبيتبعت كله.
- `.dockerignore` بيستبعد ملفات من الـ Context، وصيغته شبه `.gitignore`.
- أهم حاجات تتجاهلها: `node_modules` و `.git` و `.env` وناتج الـ build القديم.

أسئلة مراجعة

1. ليه نتجاهل `node_modules` وإحنا محتاجين المكتبات؟
2. اكتب سطر يتجاهل كل ملفات `.md` ما عدا `README.md`.
3. إزاي ملف `.dockerignore` بيحسّن استخدام الكاش؟

12

الفصل التاني عشر

رفع الـ Image على Docker Hub: tag و push و login

بنينا Image على جهازنا، فاضل نرفعها على Registry علشان أي سيرفر يقدر يسحبها ويشغّلها. دي الخطوة اللي بتكمّل الدورة: build ثم push ثم pull ثم run.

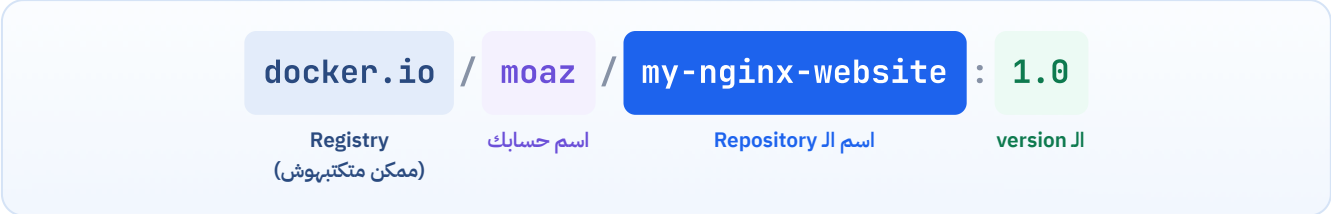
- اسم الـ Image لازم يقول هي رايحة فين
- الخطوات: tag ثم login ثم push
- Registries تانية غير Docker Hub

إضافة

الفصل ده **إضافة** على الشرح الأصلي علشان الكتاب يبقى كامل، ومكتوب بنفس الأسلوب.

12.1 اسم الـ Image لازم يقول هي رايحة فين

فاكر تشريح اسم الـ Image في فصل الـ Images؟ `REGISTRY/REPOSITORY/image_name:TAG`. علشان ترفع Image على Docker Hub، لازم اسمها يبدأ بـ **اسم حسابك** (الـ namespace)، لأن `library/` محجوزة للـ Images الرسمية بس. فلو اسم حسابك `moaz`، الـ Image لازم تبقى بالشكل ده:



شكل 12.1 اسم الـ Image اللي هترفع على حسابك في Docker Hub.

12.2 الخطوات: tag ثم login ثم push

1. **اعمل حساب** على hub.docker.com (لو معندكش). ممكن تعمل Repository من الموقع، أو تسيب الـ push يعمله لوحده أول مرة.
2. **اعمل tag للـ Image:** الـ tag مش نسخة جديدة، هو مجرد اسم تاني لنفس الـ Image (نفس الـ IMAGE ID).

```
Terminal
docker tag my-nginx-website moaz/my-nginx-website:1.0
docker tag my-nginx-website moaz/my-nginx-website:latest
```

3. **سجّل دخول** من التيرمينال:

```
Terminal
docker login
```

هيطلب منك الـ username، وبدل الـ password الأحسن تستخدم **Personal Access Token** تعمله من إعدادات حسابك.

4 **ارفع الـ Image:**

```
Terminal
moaz@Linux:~$ docker push moaz/my-nginx-website:1.0
The push refers to repository [docker.io/moaz/my-nginx-website]
5f70bf18a086: Pushed
a2b9c1d3e4f5: Mounted from library/nginx
1.0: digest: sha256:4c3f...9a1e size: 1570
moaz@Linux:~$ docker push moaz/my-nginx-website:latest
```

لاحظ إن Docker بيرفع الـ Layers اللي مش موجودة بس، والـ Layers بتاعة nginx الأصلية موجودة أصلاً على Docker Hub فمبترفعش تاني (وده تاني فايدة لفكرة الـ Layers).

5 **شغّلها من أي سيرفر** عليه Docker:

```
Terminal
docker pull moaz/my-nginx-website:1.0
docker run -d -p 80:80 moaz/my-nginx-website:1.0
```

12.3 Registries تانية غير Docker Hub

نفس الخطوات بالظبط، بس بتكتب عنوان الـ Registry في أول الاسم، وتعمل login عليه هو:

```
Terminal
docker login ghcr.io
docker tag my-app ghcr.io/moaz/my-app:1.0
docker push ghcr.io/moaz/my-app:1.0
```

ومن الأمثلة المشهورة: GitHub Container Registry (ghcr.io)، و Amazon ECR، و Google Artifact Registry، و Azure Container Registry. وتقدر تقفل الجلسة بـ `docker logout`.

Best Practice: الـ Tags

- استخدم أرقام versions واضحة (زي `1.0` و `1.1`) وارفع `latest` كمان لو حابب، بس **متعتمدش على `latest` في الـ Production**، لأنه بيتغير مع كل رفع، ومش هتعرف بالظبط أنهي نسخة شغالة.
- الـ Repository على Docker Hub ممكن يبقى **Public** (أي حد يسحبه) أو **Private**. اختار حسب المشروع.

⚠ خلي بالك

أي حاجة جوه الـ Image بترفع معاها. لو الـ Repository Public وفيه ملف `.env` أو Password في `ENV`، فكده انت نشرته للعالم. استخدم `.dockerignore` ومتحطش أسرار في الـ Dockerfile.

ملخص الفصل

- اسم الـ Image لازم يبدأ باسم حسابك: `username/repo:tag`.
- `docker tag` بيدّي اسم جديد لنفس الـ Image، و `docker login` للدخول، و `docker push` للرفع.
- الـ Layers الموجودة على الـ Registry مبترفعش تاني.
- نفس الخطوات لأي Registry تاني، بس بعنوانه في أول الاسم.

؟ أسئلة مراجعة

1. عندك Image اسمها `api` وحسابك `sara`، اكتب الأوامر اللي ترفعها بـ version `2.0`.
2. هل `docker tag` بيعمل نسخة جديدة وبياخد مساحة؟
3. ليه مش كويس تعتمد على `latest` في السيرفرات؟

13

الفصل التالت عشر

Docker Compose: شغّل المشروع كله بأمر واحد

مشروع حقيقي فيه Frontend و Backend و Database. تشغيلهم بـ docker run واحد واحد، مع الشبكة والـ Volume والـ Environment، متعب وسهل تغلط فيه. Compose بيخليك توصف كل ده في ملف واحد.

المشكلة: أوامر كتير لمشروع واحد | ملف compose.yaml | أوامر Compose الأساسية

إضافة

الفصل ده **إضافة** على الشرح الأصلي علشان الكتاب يبقى كامل، ومكتوب بنفس الأسلوب.

ملف Compose مكتوب بـ YAML

قواعد YAML (المسافات، و `key: value`، والشرطة `-` للـ Lists) اتشرحت في صفحة واحدة في كتاب APIs و JSON (فصل 21). ولو الـ Compose رفض الملف، 90% من المرات السبب مسافة في غير مكانها.

13.1 المشكلة: أوامر كتير لمشروع واحد

خلينا نتخيل إننا عايزين نشغّل المشروع اللي اتكلمنا عنه في أول الكتاب: موقع (Frontend) و API (Backend) وقاعدة بيانات. بالأوامر اللي اتعلمناها، هنحتاج نكتب حاجة زي كده:

Terminal

```
docker network create app-net
docker volume create db-data
docker run -d --name db --network app-net -v db-data:/var/lib/postgresql/data \
  -e POSTGRES_USER=app -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=appdb postgres:16-alpine
docker build -t backend ./backend
docker run -d --name backend --network app-net -p 8000:8000 \
  -e DATABASE_URL=postgres://app:secret@db:5432/appdb backend
docker build -t frontend ./frontend
docker run -d --name frontend --network app-net -p 3000:80 frontend
```

أوامر كتير، والترتيب مهم، ولو حد تاني في الفريق عايز يشغّل المشروع لازم يحفظها كلها. هنا بييجي دور **Docker Compose**: أداة بتخليك توصف كل الـ Containers والشبكات والـ Volumes في ملف **YAML** واحد، وتشغّلهم كلهم بأمر واحد.

13.2 ملف compose.yaml

بنعمل في جذر المشروع ملف اسمه `compose.yaml` (أو `docker-compose.yml` بالاسم القديم). كل Container اسمه هنا **service**:

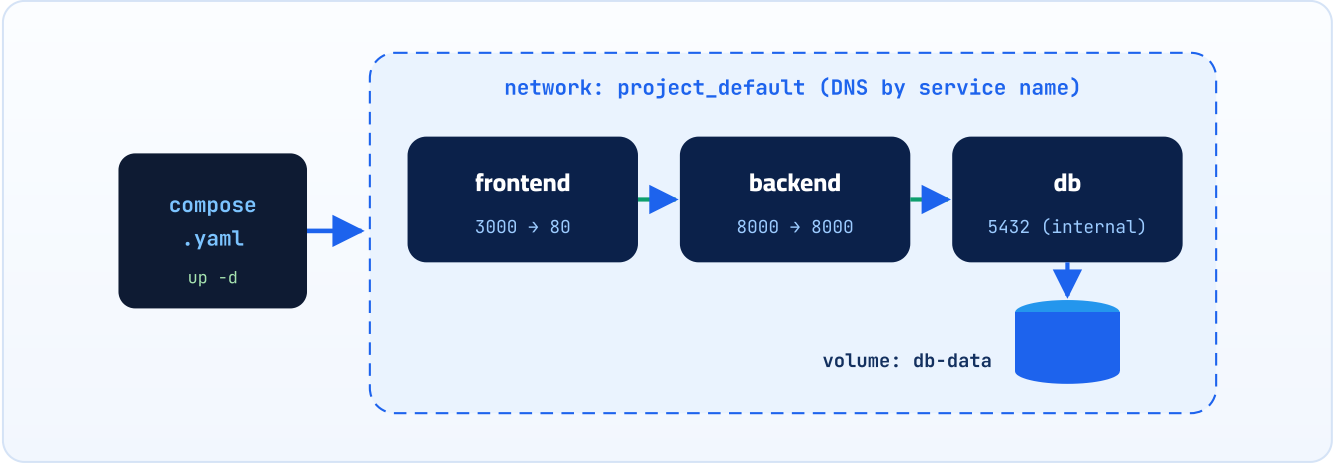
compose.yaml

```yaml
services:
  frontend:
    build: ./frontend            # يبني من ./frontend/Dockerfile
    ports:
      - "3000:80"
    depends_on:
      - backend

  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      PORT: 8000
      DATABASE_URL: postgres://app:secret@db:5432/appdb
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine    # Image جاهزة من Docker Hub
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: appdb
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d appdb"]
      interval: 5s
      retries: 5

volumes:
  db-data:
```



شكل 13.1 Compose بيعمل شبكة للمشروع، ويشغّل كل service في Container، وبيوصّل الـ Volume بقاعدة البيانات.

المفتاح	بيعمل إيه؟	الأمر اللي بيقابله
`image`	يستخدم Image جاهزة	`run image` / `docker pull`
`build`	يبني Image من فولدر فيه Dockerfile	`docker build`
`ports`	Port Mapping بصيغة `"host:container"`	`-p`
`environment`	متغيرات البيئة	`-e`
`volumes`	توصيل Volume أو Bind Mount	`-v`
`depends_on`	ترتيب التشغيل (ومع `condition` يستنى الـ healthcheck)	انت كنت بترتّب بنفسك
`healthcheck`	أمر بيتأكد إن الخدمة جاهزة فعلاً	`--health-cmd`

جدول 13.1 أهم مفاتيح ملف compose.yaml

> **الشبكة بتتعمل لوحدها**
>
> Compose بيعمل شبكة خاصة بالمشروع تلقائياً ويوصّل عليها كل الـ services، وبيشغّل الـ DNS اللي اتكلمنا عنه في فصل الشبكات. علشان كده الـ backend بيوصل لقاعدة البيانات بكلمة `db` (اسم الـ service) في `DATABASE_URL`.

13.3 أوامر Compose الأساسية

كل الأوامر دي بتتكتب في الفولدر اللي فيه `compose.yaml`:

`docker compose up -d`	يبني (لو محتاج) ويشغّل كل الـ services في الخلفية.
`docker compose up -d --build`	يعيد بناء الـ Images الأول (بعد ما تعدّل الكود أو الـ Dockerfile).
`docker compose ps`	يعرض الـ Containers بتاعة المشروع وحالتها.
`docker compose logs -f backend`	يتابع الـ logs بتاعة service معينة (من غير اسم = كل الـ services).
`docker compose exec backend sh`	يدخلك جوه الـ Container بتاع الـ service، زي `docker exec`.
`docker compose stop`	يوقّف الـ Containers من غير ما يمسحها.
`docker compose down`	يوقّف ويمسح الـ Containers والشبكة. الـ Volumes بتفضل موجودة، فالداتا مبتضيعش.
`docker compose down -v`	زي اللي فوق، بس بيمسح الـ Volumes كمان. كده بيانات قاعدة البيانات هتتمسح نهائياً.

مثال عملي: يوم شغل عادي

Terminal

```
moaz@Linux:~$ docker compose up -d
 ✔ Network shop_default       Created
 ✔ Volume "shop_db-data"      Created
 ✔ Container shop-db-1        Healthy
 ✔ Container shop-backend-1   Started
 ✔ Container shop-frontend-1  Started
moaz@Linux:~$ docker compose ps
NAME              IMAGE                SERVICE    STATUS                   PORTS
shop-backend-1    shop-backend         backend    Up 10 seconds            0.0.0.0:8000-
>8000/tcp
shop-db-1         postgres:16-alpine   db         Up 16 seconds (healthy) 5432/tcp
shop-frontend-1   shop-frontend        frontend   Up 9 seconds             0.0.0.0:3000-
>80/tcp
moaz@Linux:~$ docker compose down
```

اسم المشروع (`shop`) بياخده Compose من اسم الفولدر، وبيبدأ بيه أسماء الـ Containers والشبكة والـ Volume.

خلي بالك

- الأمر الحديث هو `docker compose` (بمسافة، جزء من Docker نفسه). الأمر القديم `docker-compose` (بشَرطة) كان أداة منفصلة ومبقاش بيتطوّر.
- المفتاح `version:` اللي بتلاقيه في أول ملفات قديمة مبقاش له لازمة في الإصدارات الحديثة، وممكن تشيله.
- `depends_on` لوحده بيرتّب التشغيل بس، مش بيستنى قاعدة البيانات تبقى جاهزة. علشان كده استخدمنا `healthcheck` مع `condition: service_healthy` .

Best Practice

متكتبش الـ Passwords صريحة في `compose.yaml` في المشاريع الحقيقية. حطها في ملف `.env` جنبه واستخدمها بـ `${POSTGRES_PASSWORD}` ، وضيف `.env` لـ `.gitignore` والـ `.dockerignore` .

ملخص الفصل

- Compose بيوصف المشروع كله (services و volumes و networks) في ملف `compose.yaml` .
- `build` و `image` و `ports` و `environment` و `volumes` بتقابل أوامر `docker` اللي اتعلمناها.
- الـ services بتكلّم بعض باسمها على شبكة المشروع التلقائية.
- `up -d` للتشغيل، و `ps` و `logs` للمتابعة، و `down` للإيقاف والمسح (و `-v` لـ Volumes بحذر).

؟ أسئلة مراجعة

1. إيه الفرق بين `image` و `build` في ملف Compose؟
2. ليه الـ backend بيكتب `db` في عنوان قاعدة البيانات بدل IP؟
3. عملت `docker compose down`، هل بيانات الـ Postgres اتمسحت؟ وماذا عن `down -v` ؟
4. عدّلت كود الـ backend، تكتب إيه علشان التعديل يشتغل؟

ملحق أ

Cheat Sheet: كل الأوامر في مكان واحد

ورقة مراجعة سريعة لكل الأوامر والـ Instructions اللي في الكتاب، مقسّمة حسب الموضوع. اطبعها وحطها جنبك وانت بتشتغل.

Images — سحب وعرض ومسح

```
docker pull nginx:1.27
```
سحب Image بـ version معينة

```
docker images
```
عرض الـ Images اللي عندك

```
docker rmi image-name
```
مسح Image (و -f بالإجبار)

```
docker image history nginx
```
عرض الـ Layers بتاعة الـ Image

```
docker image inspect nginx
```
كل تفاصيل الـ Image

```
docker image prune
```
مسح الـ Images اللي ملهاش اسم (dangling)

Build & Push — بناء ونشر

```
docker build -t app .
```
بناء Image من الـ Dockerfile في المسار الحالي

```
docker build -t app -f MyFile .
```
بناء من ملف باسم مختلف

```
docker build --build-arg V=2 -t app .
```
تمرير قيمة لـ ARG

```
docker tag app moaz/app:1.0
```
اسم جديد للـ Image قبل الرفع

```
docker login
```
تسجيل الدخول على Docker Hub

```
docker push moaz/app:1.0
```
رفع الـ Image

Run — تشغيل Container

```
docker run nginx
```
تشغيل Container (و pull لو مش موجودة)

```
docker run -d --name web nginx
```
في الخلفية وباسم واضح

```
docker run -d -p 8080:80 nginx
```
Port Mapping من الـ Host للـ Container

```
docker run -d -P nginx
```
بورت عشوائي لكل EXPOSE

```
docker run -e PORT=9090 app
```
تمرير Environment variable

```
docker run --rm -it ubuntu bash
```
Container تفاعلي بيتمسح لما تخرج

```
docker run -d --memory 512m --cpus 1 app
```
تحديد الموارد (cgroups)

Containers — إدارة

```
docker ps
```
الـ Containers الشغالة

```
docker ps -a
```
كل الـ Containers

```
docker exec -it web sh
```
الدخول جوه Container شغال

```
docker stop web
```
إيقاف

```
docker start web
```
تشغيل Container واقف

```
docker restart web
```
إعادة تشغيل

```
docker rm web
```
مسح Container (و -f بالإجبار)

```
docker container prune
```
مسح كل الـ Containers الواقفة

Monitoring — مراقبة وفحص

```
docker logs -f --tail 50 web
```
متابعة آخر الـ logs

```
docker inspect web
```
كل التفاصيل في JSON

```
docker inspect -f '{{.State.Status}}' web
```
استخراج قيمة واحدة

```
docker stats --no-stream
```
استهلاك الموارد

```
docker top web
```
الـ Processes جوه الـ Container

```
docker system df
```
المساحة المستخدمة

Volumes — تخزين البيانات

```
docker run -v my-vol:/data app
```
Named Volume

```
docker run -v /path/on/host:/app app
```
Bind Mount

```
docker run -v my-vol:/data:ro app
```
Read-only

```
docker run --tmpfs /tmp/secure app
```
tmpfs في الـ RAM

```
docker volume create my-vol
```
إنشاء Volume

```
docker volume ls
```
عرض الـ Volumes

```
docker volume inspect my-vol
```
مكانه على الـ Host (Mountpoint)

```
docker volume rm my-vol
```
مسح Volume

```
docker volume prune
```
مسح الـ Volumes اللي مش مستخدمة

Networks — الشبكات

```
docker network ls
```
عرض الشبكات

```
docker network create app-net
```
إنشاء شبكة فيها DNS

```
docker run -d --network app-net --name db
postgres
```
تشغيل على شبكة معينة

```
docker network connect app-net web
```
توصيل Container شغال بشبكة

```
docker network inspect app-net
```
تفاصيل الشبكة

```
docker run --network host nginx
```
استخدام شبكة الـ Host

```
docker network rm app-net
```
مسح شبكة

Compose — المشروع كله

```
docker compose up -d
```
تشغيل كل الـ services

```
docker compose up -d --build
```
إعادة البناء ثم التشغيل

```
docker compose ps
```
حالة الـ services

```
docker compose logs -f backend
```
logs لـ service معينة

```
docker compose exec backend sh
```
الدخول جوه service

```
docker compose down
```
إيقاف ومسح (الـ Volumes بتفضل)

```
docker compose down -v
```
ومسح الـ Volumes كمان

Dockerfile	Instructions الـ
`FROM node:18-alpine AS builder`	الـ Base Image (و AS لتسمية المرحلة)
`LABEL author="moaz"`	Metadata
`ENV PORT=8080`	متغير بيفضل جوه الـ Container
`ARG VERSION=1.0`	متغير وقت الـ build بس
`WORKDIR /app`	مسار الشغل
`COPY . .`	نسخ ملفات (الاختيار الافتراضي)
`ADD app.tar.gz /app/`	نسخ مع فك ضغط
`RUN npm install`	أمر وقت الـ build
`COPY --from=builder /app/build ./`	نسخ من مرحلة سابقة
`EXPOSE 80`	توثيق البورت
`USER appuser`	المستخدم اللي هيشغّل الـ App
`VOLUME /data`	Anonymous Volume
`ENTRYPOINT ["ping"]`	البرنامج الثابت
`CMD ["google.com"]`	الأمر أو الـ Arguments الافتراضية

⚠ أوامر خطيرة خلي بالك منها

`docker rm -f` و `docker rmi -f` و `docker volume prune` و `docker compose down -v` و `docker system prune -a`: كلهم بيمسحوا حاجات نهائياً. اقرا اللي هيتمسح قبل ما توافق.

ملحق ب

إجابات أسئلة المراجعة

حاول تجاوب على الأسئلة بنفسك الأول، وبعدين قارن إجابتك هنا. الإجابات مختصرة ومركّزة على الفكرة.

الفصل 1: المقدمة

1. الكود شغال عند المطور لأن جهازه عليه الـ dependencies والـ versions الصح، والسيرفر مختلف عنه.
2. تنافس على الموارد (app يعطّل الباقي)، وتعارض الـ versions (مساحة كبيرة واحتمال تمسح version حد تاني محتاجها).
3. الـ VM فيها Guest OS كامل، والـ Container بيشارك Kernel الـ Host.
4. العزل: Namespaces. الموارد: cgroups.
5. كانت Low-level من غير Standard ولا طريقة توزيع وإدارتها معقدة، و Docker ضاف Dockerfile و Images و Registry وتجربة سهلة.

الفصل 2: Docker

1. Dockerfile ← Image ← Registry ← Container.
2. الـ app + الـ runtime + المكتبات + الـ filesystem بتاع التوزيعة.
3. لأن كل ده بقى جوه الـ Image، والسيرفر محتاج Docker بس.
4. لأن أغلب الشغل بيبدأ من Images جاهزة وبنعدّل عليها.

الفصل 3: الـ Images

1. `docker.io/library/nginx:latest`
2. هيتأكد إن عندك آخر نسخة، ولو هي هي مش هينزّل حاجة.
3. لو فيه Container (حتى لو واقف) معمول منها.

الفصل 4: الـ Containers

1. لأن الـ Process الأساسية (sh) بتخلص على طول. الحل: `-d` مع أمر زي `sleep infinity`.
2. `ps` الشغال بس، و `ps -a` كله.
3. `docker run -d --name web -p 3000:80 nginx`
4. موجود، لأن الـ Container Layer بتتمسح مع `rm` بس مش مع `stop`.
5. لأن الـ Container بيستخدم Kernel الـ Host، و Windows محتاج Kernel بتاع Windows.

الفصل 5: المراقبة

1. `docker logs` على الـ Container.
2. `docker inspect -f` مع مسار الـ IPAddress.
3. `docker run -d --memory 256m image`

الفصل 6: الـ Layers

1. لأن الـ Image Layers مشتركة وللقراءة فقط، وكل Container بياخد طبقة رقيقة بس.
2. Copy-on-Write: نسخة بتتعمل في الـ Container Layer والتعديل بيتم عليها.
3. لأ، اتعمل Whiteout بس والملف لسه في الـ Image Layer.

الفصل 7: الـ Volumes

1. الأول Named Volume بيديره Docker، والتاني Bind Mount لمسار على جهازك.
2. Bind Mount.
3. `/var/lib/docker/volumes/NAME/_data`
4. لو اتنين حفظوا نفس الملف، اللي حفظ متأخر تعديله هو اللي يفضل ويمسح التاني.
5. بإضافة `:ro` زي `-v data:/app:ro`.

الفصل 8: الشبكات

1. لأن الـ IP بيتغير كل ما الـ Container يتعمل من جديد.
2. الـ User-defined فيها DNS داخلي بالأسماء، والافتراضية لأ.
3. لأ، لأنه بيستخدم شبكة الـ Host مباشرة.
4. `db:5432` (بورت الـ Container).

الفصل 9: الـ Dockerfile

1. لأنه بيوثّق الخطوات، وتقدر تكرّرها وتراجعها وتعدّلها.
2. `stall -y git && rm -rf /var/lib/apt/lists/*`
3. `ping 8.8.8.8`، لأن كلامك استبدل الـ CMD واتضاف للـ ENTRYPOINT.

4. لأن EXPOSE توثيق بس، والحل `-p 3000:3000` .
5. ENV بيفضل جوه الـ Container (إعدادات التشغيل)، و ARG وقت الـ build بس (زي version أداة).
6. لأن لو حد اخترق الـ App ياخد صلاحيات root وممكن يوصل للـ Host.

الفصل 10: البناء

1. المسار الحالي هو الـ Build Context.
2. لأننا ورثنا الـ CMD من `nginx:alpine` .
3. لأ، المرحلة الأخيرة بس هي اللي بتبقى Image، وهي مبنية على nginx.
4. علشان الكاش: الـ install ميتعادش إلا لو `package.json` اتغيّر.

الفصل 11: .dockerignore

1. لأن الـ Image بتعمل `npm install` بنفسها، والنسخة اللي عندك تقيلة وممكن تكون لنظام مختلف.
2. `*.md` وبعده `!README.md`
3. تغييرات في ملفات متجاهلة (زي `.git`) مبتغيرش الـ Context، فـ `COPY . .` بيفضل من الكاش.

الفصل 12: Docker Hub

1. `docker tag api sara/api:2.0` ثم `docker login` ثم `docker push sara/api:2.0`
2. لأ، مجرد اسم تاني لنفس الـ Image ID.
3. لأنه بيتغير مع كل رفع، فمش هتعرف أنهي نسخة شغالة ولا ترجع لنسخة قديمة بسهولة.

الفصل 13: Compose

1. `image` بيستخدم Image جاهزة، و `build` بيبنيها من Dockerfile.
2. لأن Compose بيعمل شبكة فيها DNS بأسماء الـ services.
3. `down` : لأ، الـ Volume موجود. `down -v` : أيوه، اتمسحت.
4. `docker compose up -d --build`

بالتوفيق في المذاكرة

أحسن طريقة تثبّت بيها اللي اتعلمته: افتح التيرمينال وجرّب كل مثال بإيدك.

MOAZ ABDEL-FATTAH